HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF ELECTRICAL AND ELECTRONICS

BACHELOR

FINAL PROJECT

Thesis Title:

DEVELOPMENT OF AN INTERACTIVE TOOL FOR SIGNALING PROCEDURE TESTS OF 5G CORE PLATFORMS

Student's Full name: HO TUAN TU
Class ET-E4 01 - K66
Lecturer: PhD. PHUNG THI KIEU HA
Reviewer:

Hanoi, June 30, 2025

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF ELECTRICAL AND ELECTRONICS

BACHELOR

FINAL PROJECT

Thesis Title:

DEVELOPMENT OF AN INTERACTIVE TOOL FOR SIGNALING PROCEDURE TESTS OF 5G CORE PLATFORMS

Student's Full name: HO TUAN TU
Class ET-E4 01 - K66
Lecturer: PhD. PHUNG THI KIEU HA
Reviewer:

Hanoi, June 30, 2025

ĐÁNH GIÁ QUYỂN ĐỒ ÁN TỐT NGHIỆP

(Dùng cho giảng viên hướng dẫn)

Tên giảng viên đánh giá: .....

Họ và tên sinh viên: ..... MSSV: .....

Tên đồ án: .....

.....

Chọn các mức điểm phù hợp cho sinh viên trình bày theo các tiêu chí dưới đây:

Rất kém (1); Kém(2); Đạt(3); Giỏi(4); Xuất sắc(5)

Có sự kết hợp giữa lý thuyết và thực hành (20)						
1	Nêu rõ tính cấp thiết và quan trọng của đề tài, các vấn đề và các giả thuyết (bao gồm mục đích và tính phù hợp) cũng như phạm vi ứng dụng của đồ án	1	2	3	4	5
2	Cập nhật kết quả nghiên cứu gần đây nhất (trong nước/quốc tế)	1	2	3	4	5
3	Nêu rõ và chi tiết phương pháp nghiên cứu/giải quyết vấn đề	1	2	3	4	5
4	Có kết quả mô phỏng/thực nghiệm và trình bày rõ ràng kết quả đạt được	1	2	3	4	5
Có khả năng phân tích và đánh giá kết quả (15)						
5	Kế hoạch làm việc rõ ràng bao gồm mục tiêu và phương pháp thực hiện dựa trên kết quả nghiên cứu lý thuyết một cách có hệ thống	1	2	3	4	5
6	Kết quả được trình bày một cách logic và dễ hiểu, tất cả kết quả đều được phân tích và đánh giá thỏa đáng	1	2	3	4	5
7	Trong phần kết luận, tác giả chỉ rõ sự khác biệt (nếu có) giữa kết quả đạt được và mục tiêu ban đầu đề ra đồng thời cung cấp lập luận để đề xuất hướng giải quyết có thể thực hiện trong tương lai	1	2	3	4	5
Kỹ năng viết quyển đồ án (10)						
8	Đồ án trình bày đúng mẫu quy định với cấu trúc các chương logic và đẹp mắt (bảng biểu, hình ảnh rõ ràng, có tiêu đề, được đánh số thứ tự và được giải thích hay đề cập đến; căn lề thống nhất, có dấu cách sau dấu chấm, dấu phẩy v.v.), có mở đầu chương và kết luận chương, có liệt kê tài liệu tham khảo và có trích dẫn đúng quy định	1	2	3	4	5
9	Kỹ năng viết xuất sắc (cấu trúc câu chuẩn, văn phong khoa học, lập luận logic và có cơ sở, từ vựng sử dụng phù hợp v.v.)	1	2	3	4	5
Thành tựu nghiên cứu khoa học (5) *(chọn 1 trong 3 trường hợp)*						
10a	Có bài báo khoa học được đăng hoặc chấp nhận đăng/Đạt giải SVNCKH giải 3 cấp Viện trở lên/Có giải thưởng khoa học (quốc tế hoặc trong nước) từ giải 3 trở lên/Có đăng ký bằng phát minh, sáng chế	5				
10b	Được báo cáo tại hội đồng cấp Viện trong hội nghị SVNCKH nhưng không đạt giải từ giải 3 trở lên/Đạt giải khuyến khích trong các kỳ thi quốc gia và quốc tế khác về chuyên ngành (VD: TI contest)	2				
10c	Không có thành tích về nghiên cứu khoa học	0				
Điểm tổng		/50				
Điểm tổng quy đổi về thang 10						

Nhận xét khác *(về thái độ và tinh thần làm việc của sinh viên)*

.....

.....

.....

.....

.....

.....

Ngày: ... / ... / 20...

Người nhận xét

(Ký và ghi rõ họ tên)

ĐÁNH GIÁ QUYỂN ĐỒ ÁN TỐT NGHIỆP

(Dùng cho cán bộ phản biện)

Giảng viên đánh giá: .....

Họ và tên sinh viên: ..... MSSV: .....

Tên đồ án: .....

.....

Chọn các mức điểm phù hợp cho sinh viên trình bày theo các tiêu chí dưới đây:

Rất kém (1); Kém(2); Đạt(3); Giỏi(4); Xuất sắc(5)

Có sự kết hợp giữa lý thuyết và thực hành (20)						
1	Nêu rõ tính cấp thiết và quan trọng của đề tài, các vấn đề và các giả thuyết (bao gồm mục đích và tính phù hợp) cũng như phạm vi ứng dụng của đồ án	1	2	3	4	5
2	Cập nhật kết quả nghiên cứu gần đây nhất (trong nước/quốc tế)	1	2	3	4	5
3	Nêu rõ và chi tiết phương pháp nghiên cứu/giải quyết vấn đề	1	2	3	4	5
4	Có kết quả mô phỏng/thực nghiệm và trình bày rõ ràng kết quả đạt được	1	2	3	4	5
Có khả năng phân tích và đánh giá kết quả (15)						
5	Kế hoạch làm việc rõ ràng bao gồm mục tiêu và phương pháp thực hiện dựa trên kết quả nghiên cứu lý thuyết một cách có hệ thống	1	2	3	4	5
6	Kết quả được trình bày một cách logic và dễ hiểu, tất cả kết quả đều được phân tích và đánh giá thỏa đáng	1	2	3	4	5
7	Trong phần kết luận, tác giả chỉ rõ sự khác biệt (nếu có) giữa kết quả đạt được và mục tiêu ban đầu đề ra đồng thời cung cấp lập luận để đề xuất hướng giải quyết có thể thực hiện trong tương lai	1	2	3	4	5
Kỹ năng viết quyển đồ án (10)						
8	Đồ án trình bày đúng mẫu quy định với cấu trúc các chương logic và đẹp mắt (bảng biểu, hình ảnh rõ ràng, có tiêu đề, được đánh số thứ tự và được giải thích hay đề cập đến; căn lề thống nhất, có dấu cách sau dấu chấm, dấu phẩy v.v.), có mở đầu chương và kết luận chương, có liệt kê tài liệu tham khảo và có trích dẫn đúng quy định	1	2	3	4	5
9	Kỹ năng viết xuất sắc (cấu trúc câu chuẩn, văn phong khoa học, lập luận logic và có cơ sở, từ vựng sử dụng phù hợp v.v.)	1	2	3	4	5
Thành tựu nghiên cứu khoa học (5) *(chọn 1 trong 3 trường hợp)*						
10a	Có bài báo khoa học được đăng hoặc chấp nhận đăng/Đạt giải SVNCKH giải 3 cấp Viện trở lên/Có giải thưởng khoa học (quốc tế hoặc trong nước) từ giải 3 trở lên/Có đăng ký bằng phát minh, sáng chế	5				
10b	Được báo cáo tại hội đồng cấp Viện trong hội nghị SVNCKH nhưng không đạt giải từ giải 3 trở lên/Đạt giải khuyến khích trong các kỳ thi quốc gia và quốc tế khác về chuyên ngành (VD: TI contest)	2				
10c	Không có thành tích về nghiên cứu khoa học	0				
Điểm tổng		/50				
Điểm tổng quy đổi về thang 10						

Nhận xét khác của cán bộ phản biện

.....

.....

.....

.....

.....

.....

Ngày: ... / ... / 20...

Người nhận xét

(Ký và ghi rõ họ tên)

PREFACE

The fast development of mobile communication technologies beyond 5G and 6G demands the utilization of open-source platforms for accelerating innovation and facilitating flexible experimentation. However, the complexity of 5G architectures imposes grand challenges on the correctness and reliability of signaling procedures, which can be initiated by both User Equipment (UE) and different core network functions.

Inspired by this challenge, the objective of this thesis is to realize an interactive client-server application to test signaling procedures in open-source 5G implementations. The tool aims to grant developers and testers immediate interaction, control, and observation over signaling dynamics in a 5G environment.

The workflow included all the stages, ranging from design and implementation of the system to integration and testing. The resulting system proves its worth in the form of a collection of representative test cases that conduct complex and realistic 5G signaling workflows.

Throughout the process of conducting this thesis project, I have faced problems and minor mistakes in my work. I am absolutely thankful for any constructive feedback and suggestions from my lecturers. I would like to express my gratitude towards Dr. Phung Thi Kieu Ha and Dr. Thai Quang Tung for their comprehensive assistance and guidance during this project. Their advice and comments have been vital to the success of this graduation thesis..

DECLARATION

I am HO TUAN TU, student ID 20213646, a member of the Advanced Program in Electronics and Telecommunication Engineering, cohort K66. My academic supervisor is Dr. PHUNG THI KIEU HA. I hereby declare that the entire content presented in the thesis titled "DEVELOPMENT OF AN INTERACTIVE TOOL FOR SIGNALING PROCEDURE TESTS OF 5G CORE PLATFORMs" is the result of my own study and research. All data presented in this thesis are entirely truthful and accurately reflect the actual measurement results. All cited information complies with intellectual property regulations, and the referenced documents are clearly listed. I take full responsibility for the content of this thesis.

Hanoi, June 30, 2025

Declarator

HO TUAN TU

TABLE CONTENT

LIST OF SYMBOLS AND ABBREVIATIONS i

LIST OF FIGURES ii

LIST OF TABLES iii

LIST OF LISTINGS iv

ABSTRACT v

CHAPTER 1. INTRODUCTION 1

1.1 Backround, Open-Source Platform 1

1.2 Problem Statement 2

1.3 Objectives 3

1.4 Scope and Limitations 4

1.4.1 Scope 4

1.4.2 Limitations 4

1.5 Structure of the Thesis 5

CHAPTER 2. TECHNICAL BACKGROUND 7

2.1 General 5G Core Network Architecture 7

2.2 Signaling Procedures in 5G 10

2.2.1 Registration Procedure 11

2.2.2 Deregistration Procedure 13

2.2.3 PDU Session Establishment 15

2.2.4 Handover 17

2.3 Open-Source 5G Projects 22

2.4 Testing Challenges 24

CHAPTER 3. SYSTEM DESIGN 27

3.1 System Architecture 27
3.2 Usecase Analysis 29
3.2.1 Actors and Goals 29
3.2.2 Narrative Use-case Descriptions 30
3.2.3 Command Context Template 36
3.3 Component Description 39
3.3.1 Interactive Tool in 5G Core 39
3.3.2 Interactive Tool in Mobile State Simulator 43
3.4 Integration Strategy 45
3.5 Implementation 47
3.5.1 Development Environment 47
3.5.2 Implementation Details-Interactive CLI in 5G Core 47
3.5.3 Implementation Details-Interactive CLI in Mobile State Simulator 50

CHAPTER 4. TESTING AND EVALUATION 53

4.1 Setup the environment 53
4.2 Test Scenarios 59
4.2.1 Execution Workflow and Testing Procedures 59
4.2.2 UE Registration 59
4.2.3 UE Deregistration 62
4.3 Evaluation Criteria 66

CONCLUSION 67

General Conlusion 67

Potential Extensions 67

REFERENCES 68

LIST OF SYMBOLS AND ABBREVIATIONS

NF	Network Functions
AMF	Access and Mobility Management Function
SMF	Session Management Function
UPF	User Plane Function
SBA	Service-Based Architecture
B5GC	Beyond 5G Core
MSSim	Mobile State Simulator
CLI	Command Line Interface

LIST OF FIGURES

Figure 2.1 General 5G Core Architecture 7
Figure 2.2 UE Registration Procedure 12
Figure 2.3 UE Deregistration Procedure 13
Figure 2.4 PDU Session Establishment Procedure 15
Figure 2.5 Xn-Handover Procedure 18
Figure 2.6 N2-Handover Procedure 20
Figure 3.1 Overall Architecture of the System 27
Figure 3.2 General Usecase Analysis 30
Figure 3.3 PDU Session Release 35
Figure 3.4 The component Interactive Tool in 5G Core 41
Figure 3.5 Component view of the interactive testing tool. 43
Figure 3.6 Flow diagram of the integration strategy. 46
Figure 3.7 Package Layer Architecture of Client/OAM Server in ETRIB5GC 48
Figure 3.8 Block diagram of the client–server signalling path within MSSim. 50
Figure 4.1 Order of System to be setup 53
Figure 4.2 UE Registration Test case 60
Figure 4.3 Result of UE Registration Success into Core Network 61
Figure 4.4 UE Deregistration Test case 63
Figure 4.5 Result of UE Deregistration Success out of Core Network 64
Figure 4.6 Result of UPF and SMF release session with UE Deregistration . 65

LIST OF TABLES

Table 2.1 Open Source 5G Projects: Detailed Overview 23
Table 2.2 Open Source 5G Simulators: Comparison 24
Table 2.3 Test Case Structure 26
Table 3.1 **Major Layers and Their Roles** 28
Table 3.2 **Interface Table** 29
Table 3.3 Mapping of procedures to CLI commands 40

LIST OF LISTINGS

Table 1 Structure of Command Context 36
Table 2 Network Function Build Process 54
Table 3 AMF Configuration Example 55
Table 4 Service Mesh Deployment Sequence 55
Table 5 MSSim emulator configuration 56
Table 6 MSSim Client Deployment and Usage 58
Table 7 OAM Client Deployment and Usage 58

ABSTRACT

The rapid development of 5G networks necessitates efficient tools for testing and validating signaling procedures in complex open-source systems. This thesis aims to design and develop an interactive client-server test tool for testing signaling processes in 5G architectures with a focus on interaction between the User Equipment (UE), gNodeB (gNB), and core network functionality. The main goal is to create a system that allows testers to trigger and observe signaling processes, offering real-time feedback on network behavior and correct functioning in multiple components.

The solution consists of two significant modules: an integrated server module with 5G core network functions, also with the UE-gNB emulator and a client CLI module for launching tests and displaying results. The client interface is realized to provide interaction with the server, enabling users to test and analyze different signaling procedures. Key implementation tasks include developing context-dependent command hierarchies for each network entity (Network Functions, UE, and gNB) using libraries such as urfave/cli for command parsing. The tool will be exemplified with selected test cases, emphasizing realistic signaling scenarios in 5G networks. The utility of the tool will be demonstrated by using it to verify 5G signaling processes for correctness and robustness.

CHAPTER 1. INTRODUCTION

1.1 Backround, Open-Source Platform

With a number of improvements over earlier generations, 5G technology represents a radical change in mobile networks. In order to satisfy the increasing demand for faster speeds, more connectivity, and more effective network performance, 5G is a complete transformation rather than merely an extension of 4G. Massive machine-type communication (mMTC), ultra-reliable low-latency communication (URLLC), and enhanced mobile broadband (eMBB) are just a few of the use cases that 5G's highly adaptable and scalable architecture can support. From consumer applications to industrial IoT systems, these developments guarantee improvements in speed, latency, capacity, and connectivity.

The 5G core network, the user equipment (UE), and the gNB (Next Generation NodeB) are all essential parts of the 5G network. In order for the network to function correctly, the signaling protocols of various components are crucial. All the way from managing mobility and sessions to establishing initial connections and security procedures, these protocols govern the interaction between devices and network components. Due to the complexity and volume of signaling messages exchanged across the network, it is critical to ensure that these procedures are correctly implemented, efficient, and resilient.

In parallel with these technological advancements, open-source platforms have gained popularity as a way to speed up and innovate the development of 5G systems. For example, open-source implementations of 5G core and radio access network (RAN) functionalities are offered by platforms such as Open5GS, free5GC, and OAI (OpenAir-Interface). These platforms support developers, researchers and enterprises with rapid rapid prototyping, collaborative development, and reproductive experimentation—foundation for academic research and industrial innovation. However, despite their benefits, open-source 5G implementations often lack comprehensive and interactive tools that satisfy of detailed testing and validation of signaling procedures, which are crucial for understanding and controlling system behavior.

5G signaling procedures can be initiated by the UE or by some of the network's fundamental functions like the Access and Mobility Management Function (AMF), the Session Management Function (SMF), or the gNB. These processes include complicated interactions between several entities. Each of these processes—registration, session establishment, handover, and release—involves a series of message exchanges regulated by 3GPP specifications. It is critical to verify these processes to be accurate since even

small errors might cause service outages, decreased performance, or compliance problems.

This thesis meets this demand by suggesting the development of such a tool, therefore providing a useful solution to the difficulty of verifying signaling procedures in open-source 5G systems. As it moves toward beyond-5G and 6G networks, the program intends to achieve the more general objectives of enhancing testing efficiency, encouraging interoperability, and cultivating deeper knowledge inside the telecoms industry.

1.2 Problem Statement

5G network architectures are becoming increasingly complicated due to their modular and service-based design, which has made testing and validating signaling methods much more challenging. Reliable connectivity, session management, and mobility handling are ensured by these procedures, which specify the step-by-step interactions between network entities such the UE, gNB, and 5G Core functions (e.g., AMF, SMF, UPF). It can be difficult to detect and make little mistakes in a single communication flow because of all the protocol levels and interwoven state transitions.

While open-source 5G platforms like Open5GS, srsRAN, and OpenAirInterface are available, the methodologies used for testing signaling validation are still not standardized, and they are either too manual or not interactive enough.

In order to dynamically investigate varied and non-trivial signaling procedures, most current technologies are either restricted to static logs or pre-scripted simulation settings. The capacity for researchers and developers to conduct real-time debugging, examine intermediate network states, and evaluate protocol functionality under varying conditions.

Furthermore, testing end-to-end signaling operations is made more difficult by the lack of a single interface that allows users to engage with both simulated UEs and fundamental network functionalities. It can be challenging for testers to observe changes in context in real time, insert test inputs at runtime, or correlate signaling signals across different components without a centralized and interactive tool. The fragmented testing environment makes development take longer, lowers trust in the system's correctness, and could lead to misunderstandings of 3GPP standard compliance.

Another major obstacle to the educational and experimental usage of 5G networks is the absence of a reliable, easy-to-understand, and extensible signaling validation framework. The inability to intuitively see and alter signaling flows hinders meaningful discovery and slows down the learning curve for students, researchers, and prototypers working with open-source networks

Given these challenges, there is a clear and urgent need for an interactive testing tool that can bridge this gap—providing real-time visibility, seamless integration with open-source components, and the flexibility to simulate and validate complex 5G signaling procedures. This thesis addresses this problem by designing and implementing such a tool to support more effective experimentation, debugging, and learning in open-source 5G environments.

1.3 Objectives

This thesis's main objective is to create, design, and validate a interactive client-server tool that lets open-source 5G systems' signalling operations be observed and tested upon. By means of real-time interaction with 5G core network functions and simulated access components (UE and gNB), the tool seeks to close the present gap in dynamic, end-to-end testing support.

More especially, the thesis aims at the following main goals:

Design and Architectural Definition: Provide a modular and extensible system architecture for the interactive tool guaranteeing flawless integration with open-source 5G components such free5GC or UERANSIM. The architecture has to support scalable client-server communication, real-time data exchange, and simple scalability.

Server Module Implementation: Provide a server-side module that can be embedded into the 5G core and UE–gNB emulator. This module should expose relevant signalling environments and offer hooks to track or control message flows at several protocol layers.

Client Tool Development: Provide a user-friendly client interface able to track UE sessions, view live network status, and interact with continuous signal exchanges. The client should let testers start processes (e.g., registration, session planning), check intermediate states, and note protocol changes.

End-to- End Signaling Procedure Testing: Demonstrate the practical utility of the system through a set of realistic test cases that reflect both common and complex signaling procedures in a 5G network. These tests should validate the correctness, responsiveness, and usability of the tool.

Evaluation and Documentation: Evaluate the system's performance, flexibility, and limitations through controlled experimentation. Additionally, provide detailed documentation to support its future use, adaptation, or extension in academic and research environments.

By means of the realisation of these goals, this thesis intends to provide a functional and practical solution that improves the testing capacity of open-source 5G plat-

forms, supports academic research, and motivates additional development towards more advanced network validation tools.

1.4 Scope and Limitations

1.4.1 Scope

This thesis mostly addresses the design and implementation of an interactive testing tool for open-source 5G systems to validate signaling techniques. The scope spans the development of a client-side interface for real-time interaction and visualization as well as the server-side integration with 5G core functions and access network emulators.

Key components encompassed by the research include:

Support for 5G signals compliant to 3GPP: Based on standard 5G architecture, the tool will center on processes including UE registration, PDU session establishment and release, and handover simulation.

Integration with open-source 5G stacks: Assuming little changes to the current source code, the system will be built to operate with platforms like etrib5gc (core network) and MSSim (Mobile State Simulator) for UE/gNB behavior.

Client-server communication framework: Customizable lightweight protocols or APIs will be created to enable data flow between the client tool and embedded server components.

Interactive testing environment: The client will allow testers to trigger signaling procedures, monitor message flows, inspect session states, and verify network behaviors under test.

Validation through selected test scenarios: An evaluation of the system will be conducted using a collection of sample test cases to showcase its practical effectiveness and usability, as part of the validation process.

1.4.2 Limitations

This thesis acknowledges a number of limitations, despite the fact that it has made specific contributions, such as:

Limited protocol coverage: The first version of the tool might not support sophisticated features as network slicing, QoS enforcement, or complex mobility scenarios and will concentrate just on a handful of signaling techniques.

Performance constraints: Due to its intended use in educational and testing contexts, the tool might not handle large-scale, real-time signaling with high throughput.

Security and reliability features: Functional correctness will be prioritized in the system's initial release, but full-scale authentication, encryption, and failure recovery

procedures will not be implemented. This is to ensure dependability and security.

Testing environment limitations: Virtualized or emulated components will be used in a controlled laboratory environment for all testing and demonstration. Beyond the scope of the thesis is real-world field deployment or compatibility testing using physical base stations and UEs.

These constraints help center the thesis on creating a functionally sound, academically valuable, practically useful prototype inside a specified experimentation and research environment.

1.5 Structure of the Thesis

In this project, I focus on proposing a solution to reduce the workload required when testing signalling procedures in Open-5G Platform. The goal is also to let the signaling flows be observed and tested with interactive tool through when the 5G Core and Simulator are deployed, as shown in subsection 1.3

The 5G Core system I chose to work with in this project is the B5GC model developed by ETRI (Electronics and Telecommunications Research Institute of Korea) [1]. The B5GC system will be used for integration and development throughout the duration of this project. And the simulator for the thesis is Mobile State Simulator (MSSim)- an open-source UE-gNB emulator, which is developed by our research team, but not published yet.

This thesis is divided into four chapters, each of which is designed to provide a systematic overview of the research motivation, technical design, implementation process, and evaluation of the proposed interactive testing tool for 5G signaling procedures.

- **Chapter One: Introduction**. This chapter covers the general foundation and reason for the research. It includes the background on 5G and open-source technologies, the recognized difficulty in current signaling validation methods, the objectives of the study, the scope and constraints of the project, and the structure of the thesis.
- **Chapter Two: Technical Background**. This chapter outlines the 5G Core Network architecture and key components such as AMF, SMF, and UPF. It reviews essential signaling procedures like UE registration, session setup, and handover, and surveys open-source 5G platforms such as Open5GS and free5GC. It also discusses challenges in testing signaling flows, including synchronization and limited observability, and explains how an interactive tool can help by enabling real-time monitoring, control, and analysis across network functions.
- **Chapter Three: System Design and Implementation**. This chapter outlines the

design and development of the interactive testing tool. It begins with a use-case analysis of common 5G procedures such as UE registration, PDU session setup/release, handover, high-traffic, and abnormal scenarios—used to define command templates. The system architecture is then presented with diagrams illustrating client-server communication and integration points. Key components like the CLI, APIs, plugin system, and emulator modules are described, along with the integration strategy for open-source platforms. The implementation section details the development environment, core features, and data flow. Finally, technical challenges encountered during development and their solutions are discussed.

- **Chapter Four: Testing and Evaluation**. This chapter evaluates the tool through several test cases involving 5G signaling procedures. It describes the test setup, including emulators and execution steps, and presents logs, screenshots, and outputs to demonstrate system behavior. The results are analyzed against criteria like responsiveness, usability, and flexibility to assess the tool's effectiveness.

CHAPTER 2. TECHNICAL BACKGROUND

2.1 General 5G Core Network Architecture

The 5G Core Network (5GC) has been developed to satisfy the requirements of a very dynamic, flexible, and scalable telecommunication network infrastructure. As a cloud-native, service-based architecture, it is instrumental in addressing the requirements of high data transfer speed, low latency, and extensive device connectivity. The architecture consists of various network functions (NFs) that collaborate to offer extensive services. All network functions strive to carry out definite functions within the control and user planes of 5G architecture.

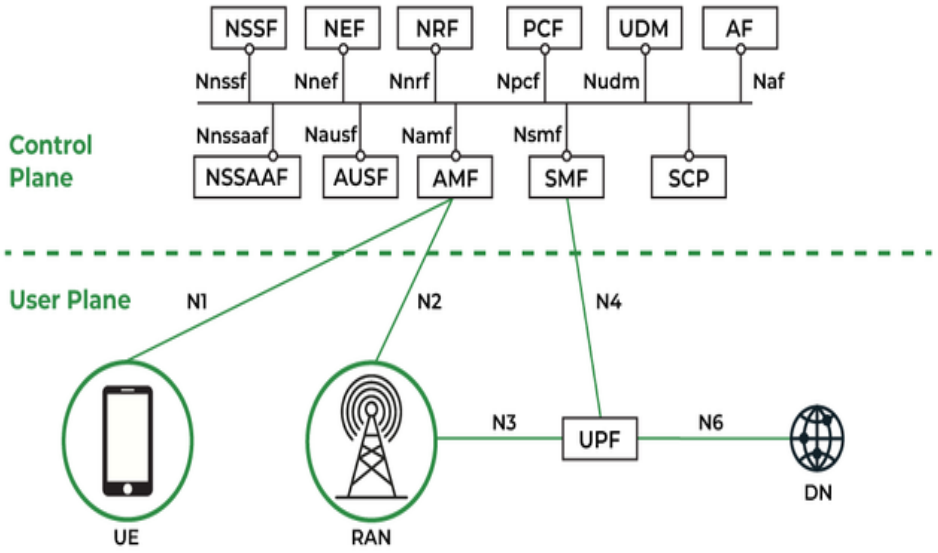

Figure 2.1 General 5G Core Architecture

[2]

From the Figure 2.1, we will cover some of the most important network functions that are the fundamental building blocks of the 5G Core Network[3]:

The Access and Mobility Function (AMF) is charged with controlling the supply of access and mobility services for User Equipment (UE) via the N1 interface. It also interacts directly with the Radio Access Network (RAN) over the N2 interface network and delivers vital services including registration, authentication, mobility management, and paging of idle devices. The AMF communicates to other network functions via service-oriented interfaces, such as relays all session management-related signaling messages between the devices and the SMF Network Function, which plays a central role in ensuring secure communication channels, thereby enabling UEs to attach effortlessly and switch from one radio cell to another.

The Session Management Function (SMF) is responsible for setting up the UE's connectivity to data networks and managing the User Plane for that connectivity. IP addresses for IP PDU sessions can be assigned by the SMF, the control function that oversees user sessions, including their creation, modification, and release. By relaying session-related messages between the devices and the SMFs, the AMF facilitates indirect communication between the SMF and the UE. The SMF interacts with other Network Functions through service based interfaces, and it also selects and controls the different UPF network functions in the network over the N4 network interface. In order to retrieve policies that it uses to configure the UPF for the PDU session, the SMF communicates with the PCF Network Function. including setting up the UPF's traffic guiding for specific sessions. In addition, the SMF gathers charging data and manages the UPF's charging features. Both online and offline charging are supported by the SMF.

The User Plane Function (UPF) has the purpose of processing and forwarding user data across the network. It is a critical element in ensuring efficient data transfer between the User Equipment (UE) and external IP networks. The UPF operates under the control of the Session Management Function (SMF), which manages the establishment, maintenance, and release of sessions. The UPF performs various types of processing of the forwarded data. It generates charging data records and traffic usage reports. It can apply "packet inspection," analyzing the content of the user data packets for usage either as input to policy decisions, or as basis for the traffic reporting. It also executes on various network or user policies, for example enforcing gating, redirection of traffic, or applying different data rate limitations.

The Network Repository Function (NRF) is a repository of the profiles of the network functions that are available in the network. The purpose of the NRF is to allow service consumers (e.g., an NF) to discover and select suitable service producers i.e., NFs and NF services without having to be configured beforehand. The NRF is updated with the new profile information whenever a new instance of a network function is deployed or altered, for example, as a result of scaling. The network function itself or another organization acting on its behalf may update the NRF profile. In order to maintain the repository and eliminate the profiles of dormant or absent network functions, the NRF also has a keep-alive mechanism.

The Unified Data Management (UDM) is responsible for managing user subscription data. It performs tasks such as access authorization, registration management, and providing authentication credentials to the AUSF (Authentication Server Function) for user authentication. The UDM also manages subscription data such as QoS profiles, roaming information, and network slice information.

The Policy Control Function (PCF) handles policy for sessions management, ac-

cess and mobility, UE access selection, PDU session selection, and future background data transfer negotiation. The PCF works with application functions and the SMF to authorise QoS and pricing for service data flows, PDU session policy control, and event reporting for PDU sessions. Access and mobility policy control, including service area limits and RFSP management, is handled by the PCF and AMF. The PCF gives the UE policy information via the AMF. This includes detection and selection policies for non-3GPP networks, session continuity mode selection, network slice selection, data network name selection, and more.

The Authentication Server Function (AUSF) offers three services and resides within the subscriber's home network. It is tasked with managing authentication within the home network, utilizing data obtained from the UE and information provided from the UDM. It offers security options to safeguard Steering of Roaming information and also to secure data throughout the UE Update operation.

To gain a clearer understanding, we can refer to the key roles performed by the 5G core [4]:

- Ensuring seamless and efficient connectivity between user devices and network services.
- Managing all aspects of user access and mobility, from registration and authentication to location tracking and connection maintenance.
- Controlling data session connections, processing and routing data flows to ensure information is transmitted quickly and accurately.
- Processing and forwarding data between user devices and the network efficiently.
- Monitoring and adjusting Quality of Service according to predefined policies, ensuring that every service meets performance and security standards.
- Implementing Quality of Service (QoS) measures to guarantee the best possible user experience.
- Performing authentication procedures to protect the network against unauthorized access.
- Managing credentials and executing authentication processes to ensure network security.
- Enabling network resource slicing to serve diverse requirements, from standard services to specialized applications demanding high quality.
- Coordinating and allocating network resources flexibly and efficiently.

Requirements and Improvements of the 5G Core

To fulfill the key roles of the 5G core effectively, several requirements and design enhancements have been introduced compared to previous generations. Only by meeting these principles can the new 5G objectives be achieved:

- **Separation of Control Plane and User Plane:** In 5G, decoupling the control plane from the user plane is critical. This allows independent management of signaling and actual data forwarding, optimizing both resource utilization and network performance. When scaling or upgrading, changes can be made without disrupting the entire system, supporting flexible expansion. User data is processed more quickly when not tied to control-plane tasks, improving the end-user experience.
- **Service-Based Architecture (SBA):** Network functions are implemented as lightweight, modular services that can be added, removed, or upgraded without impacting the rest of the system. Components communicate via standard APIs (e.g. HTTP/2 with JSON payloads), simplifying integration and interaction. SBA enables seamless integration with external services and applications, and supports continuous network evolution.
- **Network Slicing:** Virtual network slices can be created to serve different service types, each with tailored performance, security, and reliability requirements. Resources are allocated intelligently and efficiently to prevent conflicts between slices and to ensure isolation. Each slice can be optimized for specific use cases—from IoT and ultra-reliable low-latency communications to high-bandwidth applications—meeting diverse user and industry needs. Dedicated security policies per slice guarantee optimal data and service protection.

To sum up, the 5G Core Network architecture offers a microservice design that makes network management adaptable and scalable. Every network function, including the UPF for data forwarding, the SMF for session handling, and the AMF for mobility and access control, is essential to coordinate user connectivity and service delivery. Analyzing signaling procedures and creating tool that can successfully observe and validate network behavior in experimental or real-world settings require an understanding of the functions and interactions of these elements.

2.2 Signaling Procedures in 5G

Procedures are important for understanding how a telecommunication system works. This is also true in a service-based system, where different Network Functions (NFs) act as Service Consumers and Service Producers. While any NF can theoretically use a ser-

vice, building a complete and working system requires a clear description of how these services are combined to perform full operations. [5]

We have chosen a few key procedures that should give a good overview of some of the most important use cases. We have also simplified the description of the procedures to present the main components of each procedure and avoid getting bogged down into details. The description provided below should thus only be used to get an overall understanding of how each procedure works.

2.2.1 Registration Procedure

Registration is the first operation performed by the UE upon activation, allowing it to connect to the network and receive services. This procedure is essential for establishing initial communication between the UE and the network. Registration can be triggered in various scenarios and serves multiple purposes depending on the context.

- Initial Registration: utilized by the UE to establish a connection to the network following power activation.
- Periodic Registration: utilized by the UE in CM-IDLE state to indicate its continued presence to the network. The periodicity is determined by a time value obtained from the AMF.
- Mobility Registration: utilized by the UE when it exits the Registration Area or when it requires an update of its capabilities or other parameters negotiated during the Registration procedure, with or without transitioning to a new Tracking Area (TA).
- Emergency Registration: utilized by the UE for the sole purpose of registering for emergency services.

Figure 2.2 will be the example of simple registration for all purposes above. This procedure is briefly described in the following steps:

Registration Request Initiation (1-3): The UE uses the (R)AN to transmit a NAS Registration Request message to the Access and Mobility Management Function (AMF) to start the registration process. The message is forwarded if the (R)AN can map a temporary UE identity (like 5G-S-TMSI or GUAMI) to a known AMF. If not, the (R)AN forwards the NAS message to the AMF it has chosen based on the Requested NSSAI or a default AMF that has been pre-configured.

Authentication (4): Depending on the configuration of the UE and the security requirements of the network, the AMF starts the authentication process using either 5G-AKA or EAP-AKA.

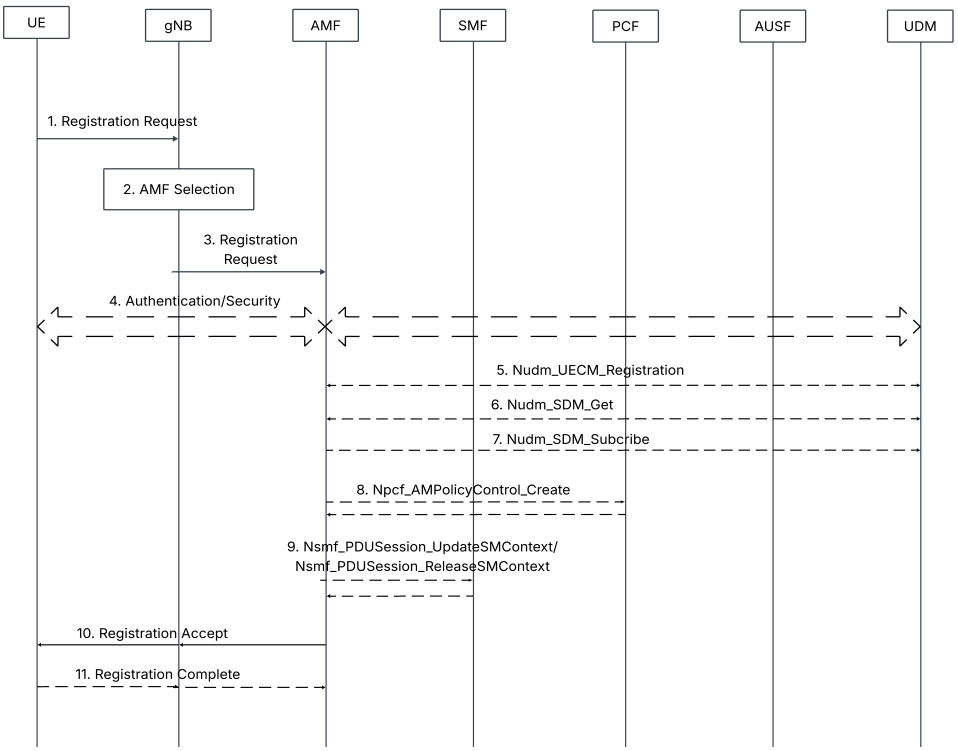

Figure 2.2 UE Registration Procedure

[5]

Handling Subscription Data and UE Context Management (5–7): Following successful authentication, the AMF uses the `Nudm_UECM` service to register itself as the serving AMF for the UE in the appropriate access network type (3GPP or non-3GPP). Additionally, it subscribes to updates via the `Nudm_SDM` service and retrieves the UE's subscription information. In this step, the previous AMF is notified of its deregistration by the Unified Data Management (UDM) function.

Access and Mobility Policy Establishment (8): If Access and Mobility (AM) management policies are in place, the AMF proceeds to establish an AM policy association with the Policy Control Function (PCF) and obtains the relevant policy rules.

PDU Session Context Handling (9): If the UE has requested to activate the User Plane for existing PDU sessions, the AMF uses the Nsmf_PDUSession_UpdateSMContext service to update the relevant session contexts. In cases where a mismatch between the UE and AMF PDU session state is detected, the AMF triggers the message of the Nsmf_PDUSession_ReleaseSMContext service to inform the Session Management Functions (SMFs) and handle the release of affected sessions.

Registration Completion (10-11): The AMF notifies the UE with a NAS Registration Accept message after the registration procedures are successfully completed. In order to verify receipt, the UE might then reply with a NAS Registration Complete

message, especially if a new 5G-GUTI or Configured NSSAI has been assigned.

UE Policy Association (12): Lastly, the AMF starts the process of creating the UE policy association with the PCF if UE-specific policies like ANDSP (Access Network Discovery and Selection Policy) or URSP (UE Route Selection Policy) are supported.

2.2.2 Deregistration Procedure

The Deregistration procedure enables the UE to notify the network that it no longer wants to access the 5G System (5GS), and likewise allows the network to inform the UE that its access to the 5GS has been revoked. As part of this procedure, the mobility and PDU session contexts associated with the UE are removed. This process may be initiated by the UE, for example, when it is being powered off. Alternatively, the network may trigger deregistration, such as when the UE fails to perform periodic registration due to being out of coverage or in response to operations and maintenance (OM) events.

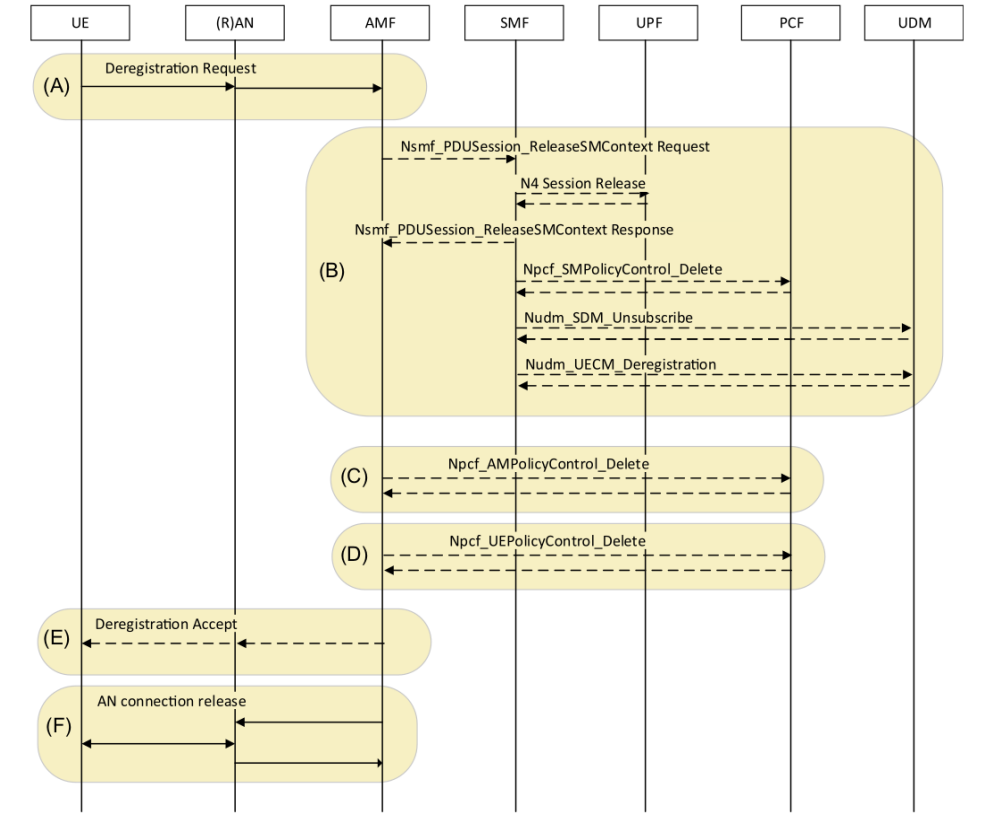

Figure 2.3 UE Deregistration Procedure

[5]

Figure 2.3 shows the full sequence of the 5G System (5GS) Deregistration procedure. In order to successfully deregister the User Equipment (UE) and properly clear all related session and policy contexts, this process entails cooperation from a number of network entities and components. To provide a clear understanding of the interactions

between the UE and the various network functions during the deregistration process, each step of the process is broken down and explained below:

Deregistration Request (A): The UE sends an NAS Deregistration Request message to AMF via the (R)AN, which informs the network that the device will no longer be using the 5G System (5GS). The deregistration process starts with this step, which also sets off further events within the network components.

Session and Policy Control Release (B): The Session Management Function (SMF) notifies each active SM context in the network after the Deregistration Request, releasing the SM context. Notifying the SMF that the relevant SM context needs to be made public is a crucial responsibility of the Access and Mobility Management Function (AMF). The SMF then takes the required steps to notify other network functions (NFs) of the PDU session's release, including the following:

- The N4 session, along with the associated User Plane resources, is released, marking the termination of the data path for the session.
- the SM policy association with the Policy Control Function (PCF) is also released, in order to guarantee that no additional policy control is applied to the session.
- For this particular PDU Session ID, the SMF deregisters from the Unified Data Management (UDM), guaranteeing that the session is eliminated from the data management layer of the system.
- Furthermore, the SMF deregisters from any future subscription data updates associated with a specific Data Network Name (DNN) and Slice Differentiated Network Slice Selection Assistance Information (S-NSSAI) if this is the final PDU session for that session.

AM Policy Control Delete: To remove the policy control rules linked to the AMF, the network components exchange the Npcf_AMPolicyControl_Delete message. This successfully completes the policy cleanup by guaranteeing that the network no longer enforces any policies pertaining to the UE's mobility and access management.

UEPoC Policy Control Delete: Similarly, the Npcf_UePolicyControl_Delete message is sent to delete any remaining user-equipment-specific policy control rules. This step ensures that all policies concerning the UE are fully removed from the network.

Deregistration Accept: When the deregistration process is successfully completed, the AMF sends a Deregistration Accept message to the UE

AN Connection Release: Finally, the AMF instructs (R)AN to release the N2 UE context. If there is still a (R)AN-level association between UE and (R)AN, the (R)AN may request the UE to release it.

2.2.3 PDU Session Establishment

The PDU Session Establishment procedure is initiated by the UE when it seeks to establish a new PDU Session or handover a PDU Session from non-3GPP access to 3GPP access. The PDU Session Establishment method is typically started by the UE, however it might be prompted by the network through the transmission of a device trigger message to an application on the UE side. Utilizing the information within the device trigger message, the application on the UE side may opt to initiate the PDU Session Establishment operation.

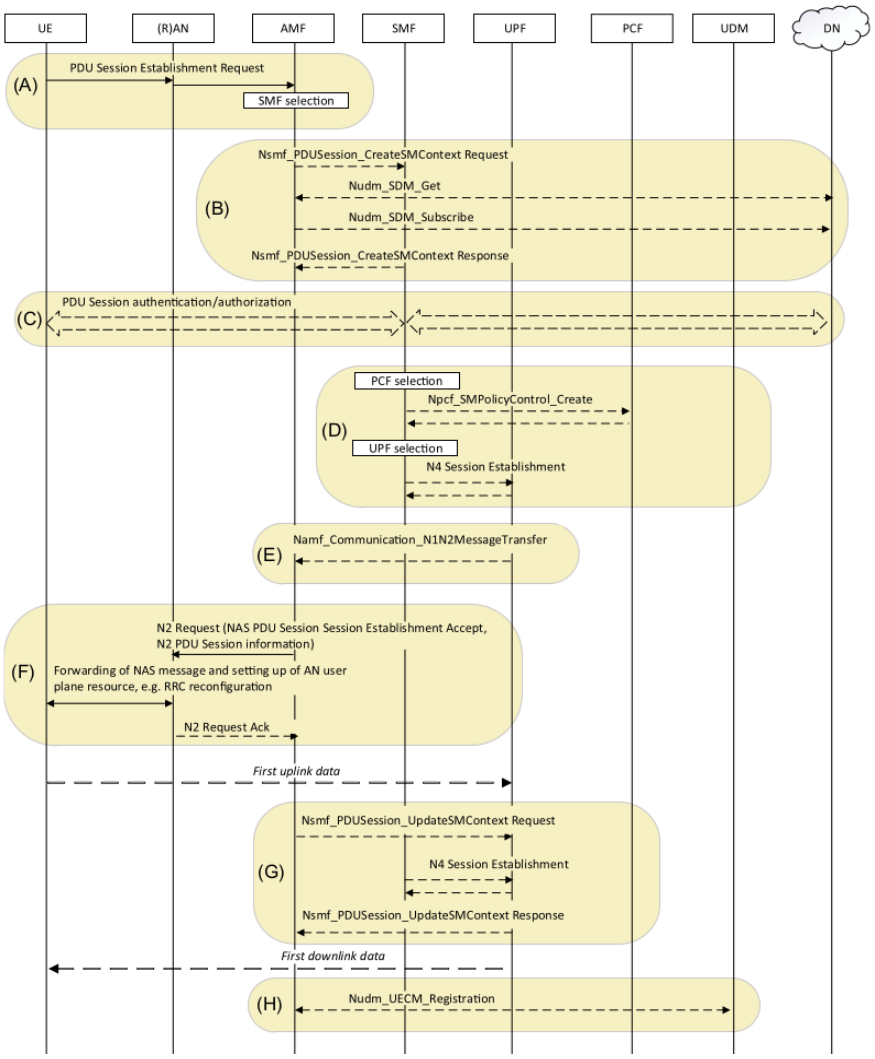

Figure 2.4 PDU Session Establishment Procedure

[5]

As shown in Figure 2.4, the diagram illustrates the entire procedure of establishing a PDU session in the 5G System (5GS). This process involves various network functions interacting with each other to ensure that the UE can successfully establish a connection, authenticate, authorize, and begin data transmission through the 5GS. From choosing the SMF (Session Management Function) to the last registration step, the process involves

several stages. In order to give a better understanding of how the 5G network components cooperate during PDU session establishment, each step of this process is examined in below:

PDU Session Establishment Request (A): The process starts when the AMF receives a 5GSM NAS (Non-Access Stratum) PDU Session Establishment message from the UE. Important details like the PDU Session ID, DNN-requested S-NSSAI, and PDU Session type are included in this message. Secure communication is ensured by the AMF processing the NAS security.

- The AMF chooses a suitable SMF if the PDU Session Establishment is a request for a new session. To find available SMFs that serve the particular DNN (Data Network Name) and S-NSSAI (Slice Differentiated Network Slice Assistance Information), the AMF may utilize the Network Repository Function (NRF).
- The AMF uses its UE context to identify the SMF that is already serving the PDU Session ID if the request involves the handover of an existing PDU Session.

Forwarding to SMF and Subscription Data Retrieval (B): The 5GSM container containing the PDU Session Establishment message is subsequently forwarded to the SMF by the AMF. In addition to retrieving the required UE subscription data pertaining to Session Management from the Unified Data Management (UDM), the SMF is in charge of managing session management tasks. To keep the session information current, the SMF also subscribes to updates on the subscription data from UDM.

Secondary Authentication (if applicable) (C): When secondary authentication is used, it is done at this point. The purpose of this step is to give the session establishment an extra degree of security.

SMF Selection of PCF, UPF, and Policy Control Session Establishment (D): The SMF then selects a Policy Control Function (PCF) and initiates the SM policy session establishment procedure to retrieve the initial set of Policy and Charging Control (PCC) rules. Additionally, the SMF selects a User Plane Function (UPF) and initiates N4 session establishment towards the chosen UPF, setting up the user plane data path.

Sending NAS PDU Session Establishment Accept (E): Upon successful session establishment, the SMF sends a 5GSM NAS PDU Session Establishment Accept message towards the UE. This message also includes GTP-U (GPRS Tunneling Protocol for User Plane) tunneling endpoint information and QoS (Quality of Service) parameters directed towards the (R)AN (Radio Access Network). The message is forwarded via the AMF to ensure proper communication between all entities.

N2 Message to (R)AN for Data Path Setup (F): The AMF then generates and

delivers the N2 message including the NAS PDU Session Establishment Accept message together with the required PDU Session data for the (R)AN. This covers QoS information as well as GTP-U tunneling data. The (R)AN establishes the necessary resources towards the UE and answers the AMF with information regarding the (R)AN GTP-U tunnel endpoint. The uplink data path is free for use once this stage is finished.

Forwarding Information from (R)AN to SMF (G): The AMF forwards to the SMF the PDU session data gathered from the (R)AN. This lets the SMF provide the UPF the (R)AN GTP-U tunnel endpoint data for downlink forwarding. By now the downlink data path is also set up and ready for data flow.

SMF Registration in UDM (H): The SMF registers itself in the UDM as the serving function for the given PDU Session ID, when the process of session establishment is effectively finished. This registration guarantees that the UDM properly handles any additional updates concerning the session and preserves the session environment.

2.2.4 Handover

Handover procedures in 5GS facilitate the transfer of a UE from a source NG-RAN node to a destination NG-RAN node. Handover processes exist with and without a Control Plane connection between the source and target NG-RAN nodes. The handover is classified as "Xn-based" since the Xn interface between the source and target NG-RAN nodes is utilized to facilitate the handover. The latter scenario is termed "N2-based" as it utilizes the N2 interface between NG-RAN and AMF for handover management. The descriptions below suggest that the Xn-based handover is less complex (using fewer messages) than the N2-based handover. It should be noted that we do not display the interactions within the NG-RAN during Xn-based handover.

2.2.4.1 Xn-based inter-NG-RAN handover

Using Xn, this process transfers a UE from a source NG-RAN to a target NG-RAN. Therefore, the Xn-based handover presupposes that the source and target NG-RAN nodes have an Xn interface. It just pertains to intra-AMF mobility; that is, Xn-handover cannot be utilized in the event that an AMF relocation is required.

An Intermediate UPF may need to be added or removed from the PDU Sessions' data stream during the process. This could occur, for instance, if the handover causes the UE to relocate to the NG-RAN from the UPF's service region, which presently possesses the N3 interface. We just emphasize the stage where the UPF procedures would be inserted if necessary in the simplest scenario below, without any I-UPF insertion, removal, or change, in order to illustrate the fundamentals of Xn hand-over and prevent the call flow from becoming complicated with specifics about N4/UPF signaling.

The whole 5GS Xn handover method is depicted in Figure 2.2. The purpose of this

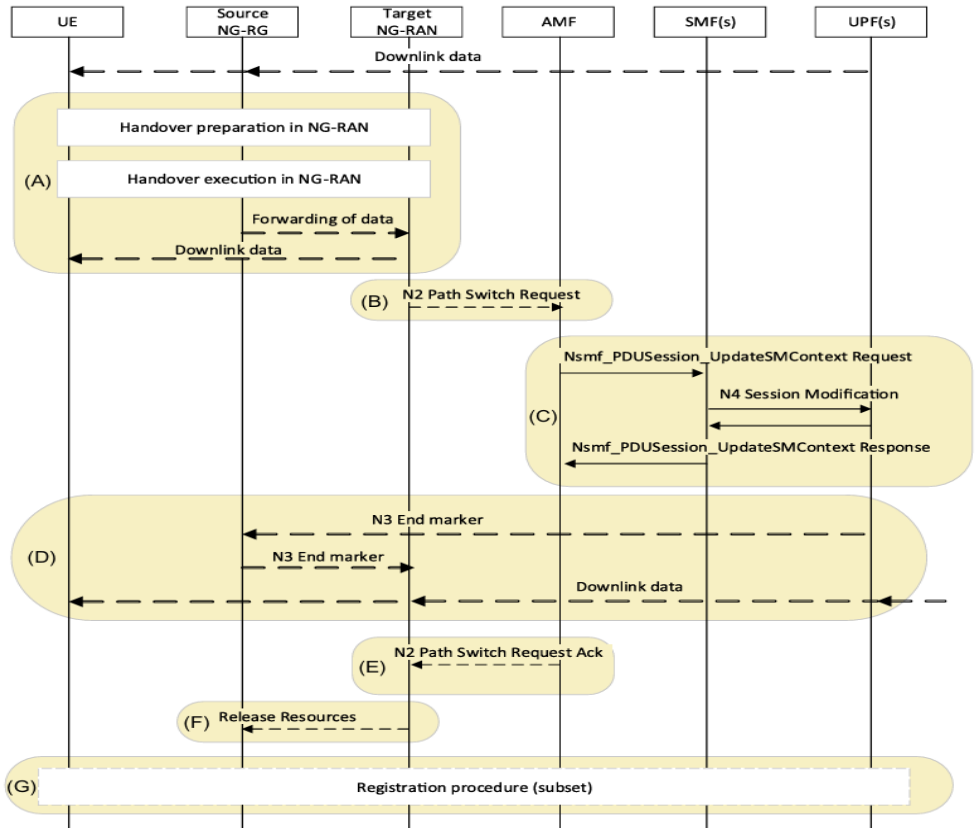

Figure 2.5 Xn-Handover Procedure

[5]

procedure is to guarantee continued service while moving by transferring a UE session between two NG-RAN nodes. A number of network functions are involved in this, such as the NG-RAN, the UPF, the SMF, and the AMF. Detailed below is an explanation of this process that incorporates the critical steps of managing resources, data forwarding, path switching, and handover preparation.

Handover Preparation and Execution in NG-RAN (A): The NG-RAN performs the required handover preparation and execution signaling prior to starting the handover process to the core network. This covers communication between the UE, the Source NG-RAN, and the Target NG-RAN. These steps guarantee that the UE is transferred to the target cell accurately.

N2 Path Switch Request (B): An N2 Path Switch Request is sent to the AMF by the Target NG-RAN after the handover has been verified in the NG-RAN. The AMF is informed by this message that the UE has relocated to a new target cell. Along with the new NG-RAN N3 tunnel information, the N2 message contains information about the PDU Sessions that are being switched. It also contains information about any PDU Sessions that the target RAN rejects, which could happen because of things like QoS

(Quality of Service) limitations in the target cell.

SMF Notification and PDU Session Management (C): Upon receiving the N2 Path Switch Request, the AMF notifies each relevant SMF that a PDU Session is affected by the handover. This includes PDU Sessions being switched to the new target cell as well as those that failed to be handled by the target cell. For PDU Sessions that are switched, the SMF provides the NG-RAN N3 tunnel information to the UPF(s), allowing downlink data to be routed to the target NG-RAN. If any PDU Sessions need to be deactivated due to the handover, the SMF initiates the PDU Session release procedure to clean up these sessions later. If there is a need to insert a new I-UPF or release an old one, these actions are performed during this step, which may involve additional N4 interactions with the relevant UPF.

Downlink Path Switching and End Marker Transmission (D): At this point, the UPF switches the downlink data path towards the new Target NG-RAN node. Downlink packets are then forwarded to the Target NG-RAN. To assist with the reordering function in the Target NG-RAN, the UPF sends one or more "end marker" GTP-U packets for each N3 tunnel on the old path (i.e., the Source NG-RAN). These end markers signal to the Target NG-RAN when the last downlink packet has been sent on the old path, allowing the forwarding tunnel between the Source and Target NG-RAN to be removed and ensuring the correct order of downlink packet delivery to the UE.

Acknowledgment of N2 Path Switch Request (E): The AMF compiles the information it has received and transmits it to the Target NG-RAN after the SMFs have responded with the required N3 tunnel data. This makes it possible for the Target NG-RAN to set up the uplink N3 data path or paths, guaranteeing that the UE can communicate without interruption in the new cell.

Release of Source NG-RAN Resources (F): The Source NG-RAN can release the resources related to the handover once the Target NG-RAN notifies it that the handover has been successfully completed and verified. By taking this step, the network's efficiency is maintained and any allocated resources can be cleaned up by the Source NG-RAN.

Mobility Registration Procedure (if applicable) (G): The UE may occasionally need to start a mobility registration process after the handover, particularly if the handover causes the UE to relocate outside of its Registration Area. This guarantees that the core network reflects the updated network context and that the UE's registration status is updated.

2.2.4.2 N2-based inter-NG-RAN handover

N2-based handover is used in case there is no Xn signaling interface between Source NG-RAN and target NG-RAN, e.g., due to deployment or implementation aspects. Here, the core network is responsible for both the handover signaling and the communication between the source and target NG-RAN. For reasons such as load balancing or new radio conditions, the source NG-RAN can opt to start a N2-based handover to the destination NG-RAN. This graphic depicts the 5G System N2-based handover proce-

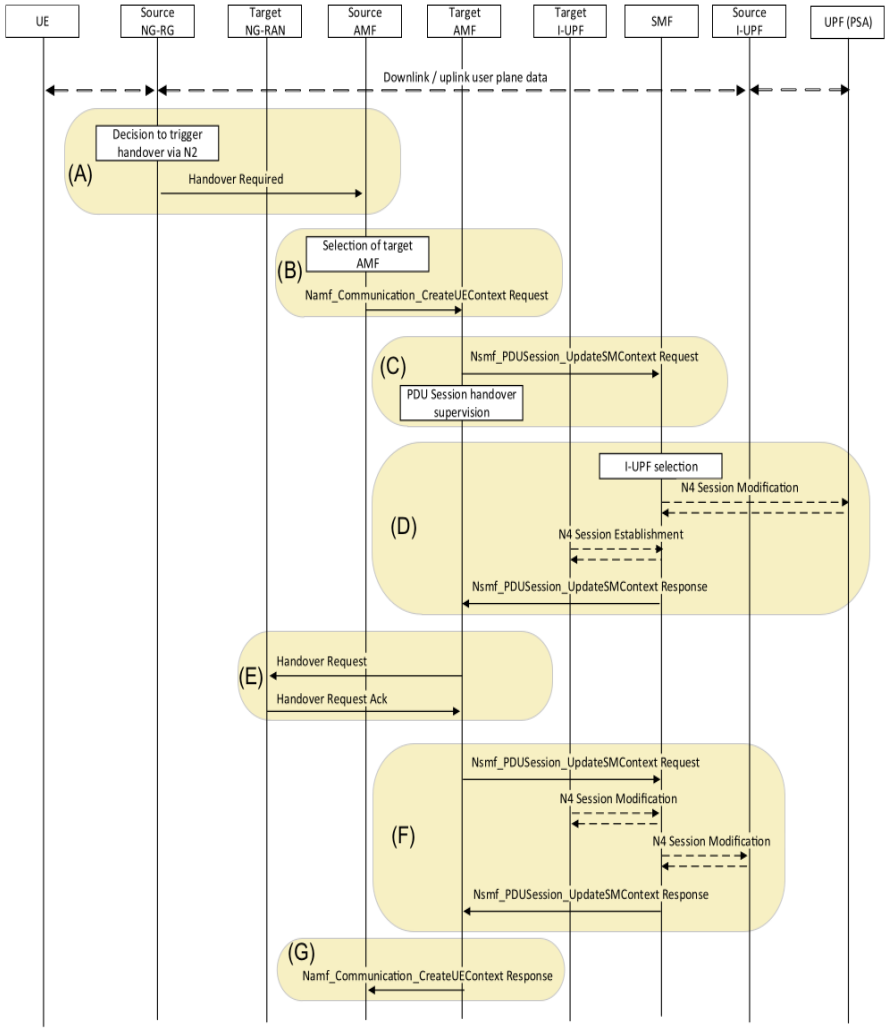

Figure 2.6 N2-Handover Procedure

[5]

dure, as depicted in Figure 2.6. To maintain data session continuity, the handover process is initiated whenever the User Equipment (UE) transitions from one NG-RAN node to another. The Source NG-RAN, the Target NG-RAN, the AMF, the SMF, and the UPF are all involved in this multi-step process. This section provides a detailed outline of the process, including the steps involved in managing the handover and the PDU sessions.

Decision to Trigger Handover via N2 (A): The process starts when Source NG-RAN chooses to set off a N2-based handover. It provides the AMF with a N2 Handover

Required message comprising vital information including the ID of the target NG-RAN and specifics on the PDU Sessions to be passed over. This initiates the handover process and indicates that the UE session has to be moved to another cell.

Selection of Target AMF (B): After getting the Handover Required message, the AMF determines if it can service the target NG-RAN, the new location of the UE. Should the AMF fail to satisfy the goal NG-RAN, it chooses an other target AMF. To set the UE's context in the new location, the AMF requests Create UE Context from the target AMF including the information acquired from Source NG-RAN.

PDU Session Handover Monitoring (C): The target AMF now interacts with the SMF(s) responsible for the impacted PDU Sessions to notify them of the handover. At this stage, the AMF monitors the responses from each SMF to ensure that all requisite actions are executed for the transfer of PDU Sessions. The AMF cannot postpone indefinitely for responses but must guarantee that the transfer progresses promptly to enable the UE to sustain service.

UPF Decision and I-UPF Selection (D): The SMF(s) determine whether the current UPF(s) can meet the new goal NG-RAN or whether a new I-UPF (Interim UPF) is needed after the SMF(s) acquire the appropriate information from the AMF. We suppose in this particular flow that a Target I-UPF must replace the Source I-UPF. The SMF generates the new Target I-UPF and responds the AMF with the revised data.

Handover Request to Target NG-RAN: The AMF sends a Handover Request to the Target NG-RAN once it has received all the necessary responses from the SMF(s) or is unable to wait any longer. The PDU Session information and relevant details for the transfer process are included in this message. The N3 tunnel information for each PDU Session that can be effectively transferred is returned by the Target NG-RAN. It also denotes any PDU Sessions that are not eligible for acceptance by the Target NG-RAN, such as those that do not satisfy the necessary QoS or other network conditions.

Forwarding PDU Session Information to SMFs: After receiving the PDU Session information from the Target NG-RAN, the AMF forwards it to the appropriate SMF(s). If indirect forwarding is necessary, the SMF establishes the requisite forwarding tunnels to transfer data from the Source NG-RAN to the Target NG-RAN via the UPF. This guarantees that the UE can continue to receive data in a seamless manner during the handover procedure.

Target AMF Response to Source AMF: On completion of the preparation phase, the Target AMF responds to the Source AMF, indicating that the handover is prepared for execution. This message also contains supplementary information that must be transmitted to the Source NG-RAN, including information on any indirect forwarding channels

and any PDU Sessions that were not established in the Target NG-RAN. These specifics will be addressed during the final execution phase.

2.3 Open-Source 5G Projects

The growth of open-source 5G initiatives in the past few years has contributed to 5G network research and experimentation. Without the need for pricey proprietary equipment, these platforms offer priceless resources for testing and replicating essential network processes. Open5GS and free5GC are two examples of open-source 5G platforms that provide developers and researchers with all the tools they need to test and evaluate 5G signaling procedures, from the most fundamental ones, like handover and registration, to the most advanced ones, like network slicing and mobility management.

Also, two widely adopted open-source simulators are UERANSIM and Package Rusher Extended. UERANSIM is a powerful tool for simulating the behavior of User Equipment (UE) and gNodeB in 5G networks, enabling detailed testing of signaling procedures such as registration, PDU session establishment, and mobility management. On the other hand, Package Rusher Extended is designed to facilitate the generation and replay of signaling traffic in customizable formats, making it useful for stress testing and benchmarking the performance of 5G core implementations. The combination of these simulators allows researchers to emulate realistic end-to-end communication scenarios without relying on commercial hardware.

Users are able to personalize network functions (NFs) and test out various 5G signaling and data transmission situations using Open5GS and free5GC due to their modular and extensible design. Academic and business research institutions use these open-source projects to investigate the pros and cons of 5G networks and create tools for usage in actual deployments.

Open5GS

Open5GS is a comprehensive, open-source C-language implementation of both the 5G Core Network (5GC) and the 4G Evolved Packet Core (EPC), fully compliant with 3GPP Release 17. It provides a full suite of modular network functions—AMF, SMF, UPF, PCF, UDM, NRF and NSSF—each deployable independently via Docker or Kubernetes. With its built-in RESTful management APIs and a flexible command-line interface, Open5GS enables rapid prototyping, private-network deployments, and end-to-end 5G testing for research, development and small-scale production use.

free5GC

free5GC is a Linux Foundation open-source project that implements the 5G core

network defined in 3GPP Release 15 and beyond. Written primarily in Go, it offers cloud-native network functions—AMF, SMF, UPF, PCF, UDM, AUSF and NRF—that can be orchestrated via Helm charts on Kubernetes. Its clean, modular design and extensive documentation make free5GC ideal for academic research, performance benchmarking, interoperability testing, and demonstrations of advanced features such as network slicing and multi-access edge computing.

Table 2.1 compares and contrasts the two most well-known open-source 5G projects, free5GC and Open5GS, outlining their respective strengths in terms of network functionality, capabilities, and important features.

Table 2.1 Open Source 5G Projects: Detailed Overview

Category	**Open5GS**	**free5GC**
Overview	Open5GS is a comprehensive open-source 5G core network solution that supports both NSA and SA deployment modes. It's designed to implement key 5G core network functions and offers a flexible environment for testing.	free5GC is an open-source, fully-compliant 5G core network, focused on research, development, and experimentation. It supports SA modes and is highly customizable with a modular architecture.
Deployment Mode	Supports both NSA (Non-Standalone) and SA (Standalone) modes.	Only SA (Standalone) mode
Testing Focus	Focus on end-to-end 5G signaling procedures, such as registration, PDU session, handover, and load tests.	Focus on complex signaling procedures and performance tests, with flexible scenario configurations.
Deployment Flexibility	Can be deployed on virtual machines or Docker containers.	Built on a microservice architecture, each NF runs in a separate container.

UERANSIM

UERANSIM is the open-source, state-of-the-art 5G UE and RAN (gNodeB) simulator. It introduces the first fully open-source standalone UE and gNodeB implementation for 5G-SA testing. UERANSIM supports key NAS and NGAP procedures—registration, security mode, PDU session setup/release, handover, paging—while simulating the lower radio layers (PHY/MAC/RLC/PDCP) over UDP for lightweight operation. It is implemented in C++ under GPL-3.0.

PackageRusher

PacketRusher is a high-performance 5G UE/gNodeB simulator and CP/UP load tester designed for automated validation of 5G Core Networks. Built in Go (with some C), it can simulate thousands of UEs and base stations in parallel, exercising both N1 (NAS) and N2 (NGAP) interfaces, capturing PCAP, and supporting advanced features like SUCI concealing/deconcealment. It integrates a mocked 5GC/AMF for end-to-end scenarios and has been stress-tested up to 10 000 UEs.

Table 2.2 presents a side-by-side comparison of UERANSIM and PacketRusher:

Table 2.2 Open Source 5G Simulators: Comparison

Category	**UERANSIM**	**PacketRusher**
Overview	Open-source 5G-SA UE and gNodeB simulator for testing 5G Core and studying system behavior.	High-performance UE/gNodeB simulator and CP/UP load tester for automated 5G Core validation.
Simulated Components	UE, gNodeB over NAS (N1), NGAP (N2), GTP-U; RRC procedures; lower layers via UDP emulation.	Multiple UEs and gNodeBs; full NAS (N1) and NGAP (N2); integrated mock AMF/5GC for end-to-end scenarios.
Key Features	Registration, deregistration, PDU session setup/release, handover, paging, service request.	SUCI concealing/deconcealment, attach/detach, PDU session lifecycle (up to 15 per UE), handover, CM-IDLE.
Implementation	C++, modular with separate control-plane and user-plane interfaces.	Go, C language with microservice CLI for scripting
Performance	Community-maintained; suitable for functional testing; not designed for large-scale stress tests.	Benchmarked up to 10 000 UEs; built-in CP/UP stress and PCAP capture.

2.4 Testing Challenges

Testing 5G signaling procedures is inherently complex. Each procedure must meet stringent timing constraints, involve multiple network functions (NFs), and traverse several protocol layers. A single out-of-order message or an expired timer can cascade through the AMF, SMF, UPF and the RAN/UE emulators, making it difficult to pinpoint the root cause. On top of that, 5G's service-based interfaces introduce dynamic, asynchronous interactions—for example, handover and PDU session establishment depend on real-time radio conditions—so testers must emulate diverse states and correlate

message sequences that are not simple request–response pairs.

Moreover, core functions and the RAN emulator often come from different vendors. Even though they adhere to the same 3GPP standards, subtle implementation differences can break interoperability. As a result, comprehensive validation requires not only functional correctness but also cross-vendor compatibility testing and fault injection across all components.

To make tests reproducible and unambiguous, each scenario must be parameterized with precise inputs (e.g. UE IMSI for registration, target gNB ID for handover) and a clear definition of both expected behavior and failure conditions. Table 2.3 summarizes this structure. Each row represents a distinct test case, with columns for:

- **Input:** The stimuli injected into the system (e.g. messages, identifiers).
- **Expected Output:** The success criteria (e.g. state transitions, acknowledgements).
- **Error Report:** The specific anomalies to detect if the test fails (e.g. mismatches, timeouts).

This organization ensures coverage of both nominal and stress conditions, while explicitly documenting what constitutes an anomaly in each scenario.

During execution, rich telemetry must be collected from every NF and emulator: detailed message traces with timestamps and sequence numbers; snapshots of UE and session states; timer expirations and retransmission counters; resource metrics (CPU, memory, queue depth under load); and logs of protocol or radio-link failures. Capturing and correlating this manually is time-consuming and error-prone. An interactive client–server testing tool streamlines the process by:

- Providing a unified CLI to inject inputs and drive scenarios.
- Streaming real response, state snapshots and performance metrics from all components.
- Automatically flagging deviations from expected outputs.

Testers can thus focus on diagnosing root causes instead of data gathering, iterate rapidly, and gain confidence that complex 5G signaling procedures truly conform to the standards.

Table 2.3 Test Case Structure

Test Case	Input	Expected Output	Error Report
Registration	UE identity (IMSI), initial Registration Request	Success (`5GMM-REGISTERED`) / Failure	Authentication failure, IMSI mismatch
Deregistration	Deregistration Request, cause value	Context removed / NGAP Release Complete	Timer expiry, unexpected response
PDU Session Establishment	PDU Session Request, DNN/SST parameters	PDU Session Active / policy installed	Context-setup failure, resource denial
Handover	Handover Required, target gNB ID	RRC Reconfiguration Complete / SMF update	Radio timeout, wrong target selection
Load Test	Multiple concurrent UE registrations	All UEs connected or failed	Overload errors, packet loss
Traffic Test	Continuous data flow (uplink/downlink)	Throughput meets threshold	Packet loss, throughput drop

CHAPTER 3. SYSTEM DESIGN

3.1 System Architecture

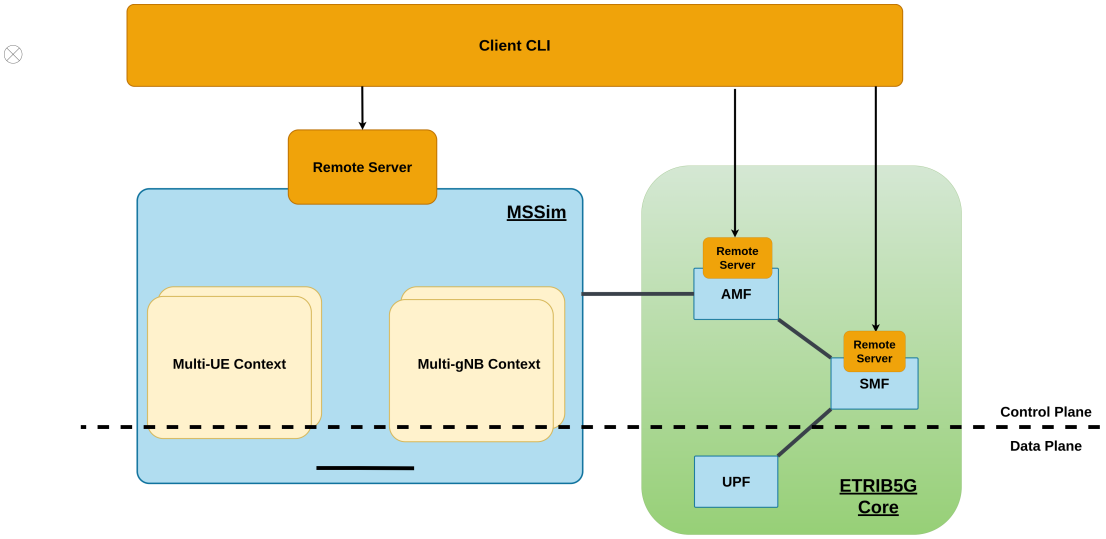

Figure 3.1 Overall Architecture of the System

Figure 3.1 depicts the layered, plug-in–centric architecture adopted for the interactive 5G signaling procedures. At a high level, the design separates (i) a Client CLI that orchestrates tests and renders live telemetry, (ii) a Remote Server that brokers commands and measurements, (iii) an emulator block (MSSim) that generates multi-UE and multi-gNB traffic, and (iv) the ETRIB5G Core [1] NFs equipped with lightweight plug-ins. The dotted line in the diagram distinguishes the control plane—where signaling messages flow and are inspected—from the data plane, which remains unmodified to preserve realistic user-plane behavior.

Table 3.1 Major Layers and Their Roles

Layer	Primary Building Blocks and Responsibilities
Client CLI	• CLI interface for selecting one of six canonical procedures (registration, deregistration, PDU setup, PDU release, hand-over, load/abnormal test) • Template editor for parameterising IMSIs, target-gNB IDs, traffic profiles • Real-time dashboards for message sequences and KPIs with alerting
Remote-Server	• Secure HTTP endpoint that authenticates clients • Session cache and ID management • Message broker that fan-outs commands to MSSim and NF plug-ins and aggregates their telemetry
MSSim (Emulator Block)	• Multi-UE context (hundreds–thousands of IMSIs, mobility models) • Multi-gNB context (RAN connection management, PDU Session Handling) • Traffic generators for load and stress scenarios
ETRIB5GC Core	• AMF, SMF, UPF binaries • Side-car plug-ins that subscribe to internal event buses and export NAS, NGAP, timer, and resource metrics

Table 3.1 clarifies how all the system is decomposed into four logical layers, each serving a well-defined purpose. The Client CLI concentrates every user-facing function—procedure selection, parameter editing, and real-time dashboards, so testers can drive experiments without touching back-end code. The Remote Server introduces a thin control plane that conducts sessions, multiplexes commands, and aggregates telemetry; this keeps orchestration logic isolated from both the simulator and the core. The blue MSSim block supplies scale and realism: by modelling thousands of virtual UEs and multiple gNBs, it generates the traffic patterns needed for load, mobility, and support for signaling procedure tests. Finally, the ETRIB5G Core contains unmodified NF binaries instrumented only by lightweight plug-ins, ensuring that signalling paths remain standards-compliant with the simulator. Taken together, these layers establish a clean separation of concerns—user interaction, emulation, and protocol processing—allowing the system to be extended or debugged one layer at a time.

Table 3.2 Interface Table

Interface	Protocol	Payload/Purpose
Client - Remote Server	HTTP	JSON commands; Protobuf telemetry streams
Remote Server	Implemented directly	Low-latency fan-out of test commands; sharded telemetry topics per NF
MSSim ETRIB5G Core	3GPP N1 / N2 / N3	Unmodified RRC/NAS/NGAP signaling; user-plane GTP-U packets

Table 3.2 maps the critical interfaces that combine the layers together. Northbound traffic between the Client and Remote Server travels over secure HTTP channels, carrying JSON commands and encoded entity to visualize user-friendly interface. East–west links inside the back-end use direct integration, enabling the Remote Server to fan-out control messages to MSSim and NFs. Southbound paths remain straightforward 3GPP N1/N2/N3 interfaces, so neither the simulator nor the core acts differently from standard NAS, NGAP, or GTP-U behavior. This interface matrix therefore preserves protocol integrity on the data plane, achieves low-latency coordination on the control plane, and keeps the entire architecture independently on the Service Provider and easily portable to alternative open-source or commercial 5G stacks.

3.2 Usecase Analysis

3.2.1 Actors and Goals

A systematic identification of user goals and system responsibilities constitutes the foundation of the design presented in subsequent chapters. In the domain of 5G experimentation, those user goals for Client CLI map directly onto well-defined *signalling procedures*. Each procedure involves multiple logical actors (UE, gNB, AMF, etc.) and imposes stringent timing constraints, making an interactive test harness both useful and challenging.

The Client-Remote Server Tool therefore has to reproduce the most frequently occurring signalling situations, but at the same time must remain operable through a concise, scriptable command set. The present section therefore performs a classical software-engineering use-case analysis: actors are listed, their intentions delineated, and the interactions formalised, as shown in figure 3.2:

- UE Registration
- UE Deregistration

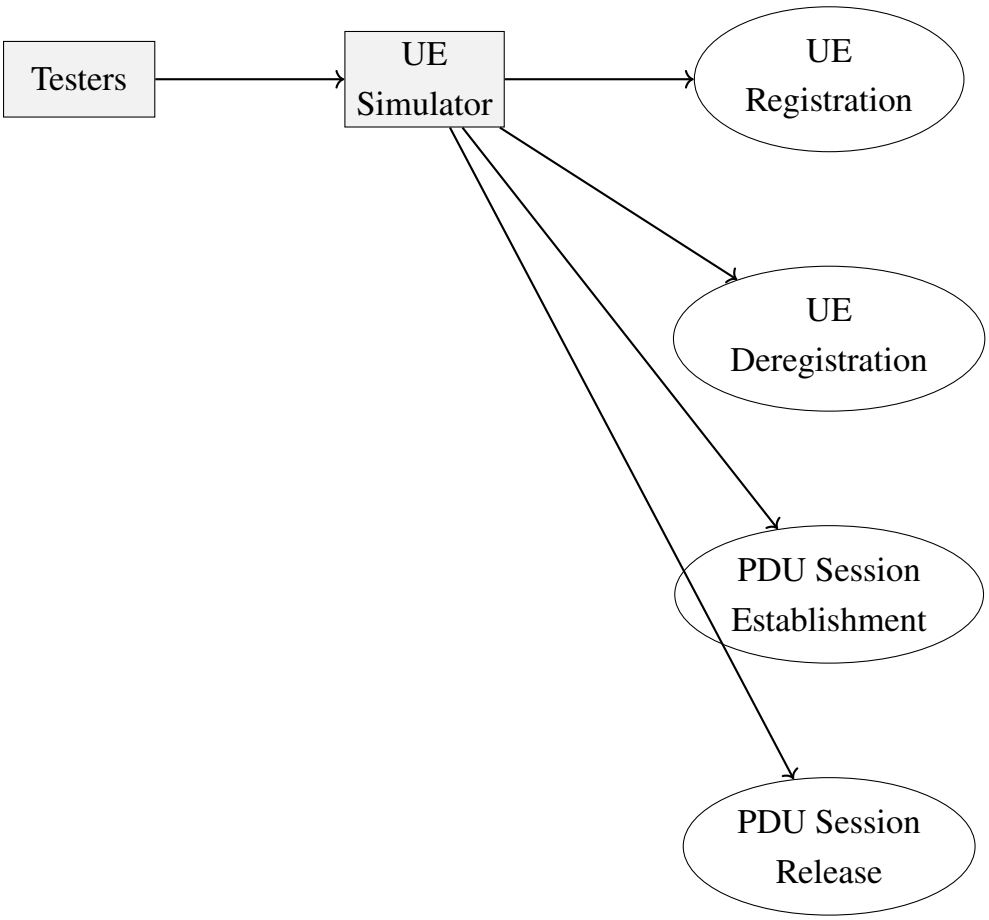

Figure 3.2 General Usecase Analysis

- PDU Session Establishment
- PDU Session Release

Each subsubsection below shows the use-case with highlights design-relevant details, and finally distils the requirements that shaped the command structure of the tool. The resulting command template provides the semantic bridge between the Mobile State Simulator (MSsim) and the 5G core network functions

3.2.2 Narrative Use-case Descriptions

The following paragraphs formalize each procedure in terms customary to requirements engineering.

3.2.2.1 Use case 1 – UE Registration.

The registration procedure marks the first contact of a detached UE with the 5G core. Its primary purpose is to authenticate the subscriber, negotiate ciphering and integrity algorithms, and finally create an AMF UE context, which acts as the anchor for all subsequent sessions.

Standards Reference (Normal Course)-Figure2.2.

1. UE sends NAS Registration Request via NG-RAN.
2. gNB selects AMF.
3. Authentication & security with AUSF/UDM.
4. AM policy creation at PCF.
5. Initial SM context create or release at SMF (zero-PDU case).
6. AMF transmits Registration Accept / Complete.

The formation of Client requirements

Functional Requirements:

- The client CLI shall allow an tester to select a UE node and issue the `|register|` command.
- The command raises the internal event `|RegisterInit|` and triggers an event of UE so that a valid 5GS *Registration Request* NAS PDU is built.
- After the request (message 1 in Fig. 2.2) the simulator performs authentication, security and policy steps (messages 4–9) and finally delivers *Registration Accept / Complete* (messages 10–11).
- The client CLI prints the result ("`|UE <id> registered successfully|`") and keeps the connection open for further commands (PDU session, service request).
- The system must queue events so that multiple UEs can be triggered concurrently.

Steps in which the client CLI can participate in the procedure

1. The tester selects a UE in the client CLI and launches the registration procedure.
2. The client CLI packages a *Registration Request* and forwards it to the radio access network, which relays the message to the core.
3. All remaining signalling (authentication, security mode, policy creation, session setup) is handled automatically; the client only displays progress information.
4. When the core returns *Registration Accept/Complete*, the client confirms successful registration and the prompt becomes available for further commands.

3.2.2.2 Use case 2 – UE Deregistration.

Deregistration is essentially the counterpart of registration, yet it exercises a wider range of internal APIs because the AMF must release potential child objects—namely

SM contexts and NG-RAN resources. In a UE-initiated scenario the message flow begins with a Deregistration Request (UE→NW) The implementation under test forwards the function for `Nsmf_PDUSession_ReleaseSMContext` to each serving SMF 2.3, which in turn triggers N4 Session Release on every attached UPF. The AMF finally sends Deregistration Accept, waits for the NGAP UE Context Release Complete, and then purges its database record.

Normal course of events-Figure 2.3

1. UE issues Deregistration Request (NAS).
2. AMF invokes `Nsmf_PDUSession_ReleaseSMContext` on each SMF.
3. SMF triggers N4 Session Release on attached UPF(s).
4. AMF sends Deregistration Accept (NAS).
5. NGAP UE Context Release Complete received by AMF.
6. AMF purges UE context and releases NG-RAN resources.

The formation of Client requirements

Functional Requirements:

1. Testers type `|deregister|` ⇒ `|DeregistraterInit|` injected into Simulator or start a network-initiated deregistration for any selected UE
2. State event creates *Deregistration Request* and sends it (step A).
3. Core releases contexts (steps B–D); the CLI only logs callbacks It will also display a UE's full AMF context before the procedure and refresh the display once the context is removed.
4. On *Deregistration Accept* (step E) the CLI outputs "`|UE <id> deregistered|`" and returns to the prompt.

Steps in which the client CLI participates in the procedure

1. The tester issues the deregistration command, optionally selecting the desired deregistration type.
2. The client CLI creates and sends a *Deregistration Request* through the access network to the core system.
3. The core releases all UE-related sessions and policies; meanwhile the client CLI shows status messages without further interaction.

4. After receiving a final *Deregistration Accept*, the client CLI reports completion and allows the operator to terminate or re-register the UE as needed.

3.2.2.3 Use case 3 – PDU Session Establishment.

Where registration prepares the ground, PDU session establishment (PDU-SE) delivers the payload: IP connectivity to a data network. For the open-source core it is the moment when the orchestration logic of SMF, PCF and UPF is exercised for the first time.

Following acceptance of the NAS → PDU Session Establishment Request, the AMF performs SMF selection via NRF discovery or static routing rules, as shown in figure 2.4 The selected SMF records the request in a temporary SM context, queries the UDM for subscribed DNN/S-NSSAI information, and subscribes to SDM updates.

Normal course of events-Figure 2.4

1. UE sends NAS PDU SESSION ESTABLISHMENT REQUEST via NG-RAN.
2. AMF performs SMF selection (via NRF or static routing).
3. SMF records request in temporary SM context; queries UDM for subscribed DNN/S-NSSAI and subscribes to SDM updates.
4. If policy triggers apply, SMF contacts PCF to refine QoS, GBR or charging parameters.
5. SMF allocates an IP address and instructs UPF via N4 Session Establishment to create GTP-U tunnels.
6. SMF returns NAS PDU SESSION ACCEPT with PDU_Address, S-NSSAI, DL-TEID, etc.

The formation of Client requirements

Functional Requirement

- The tester will be able to issue a *create-session* command for a chosen UE (or UE group) by means of the interactive client.
- The client will translate this command into an HTTP POST payload and send it to the back-end simulator with suitable endpoint.
- The UE state machine will maintain one finite-state machine (FSM) per PDU session (*Inactive* → *Active-Pending* → *Active*).

- The tool will expose audit commands such as *list-ue* and *list-sessions* so that the tester can verify whether the establishment succeeded or failed.

Steps in which the client CLI can participate in the procedure

1. The operator triggers a *PDU Session Creation* command and, if required, specifies parameters such as slice identifier or DNN.
2. The client serialises the request as a JSON `CommandRequest` and posts it to the back-end REST interface of the simulator
3. The UE state-event generates the NAS *PDU Session Establishment Request*; the 5G core proceeds with resource allocation while the client only displays intermediate status messages.
4. When the NAS *Establishment Accept* or *Reject* arrives, the client reports the final outcome and makes follow-up commands (*list-sessions*, *release-session*, ...) available to the operator.

3.2.2.4 Use case 4 – PDU Session Release.

PDU Session Release cleans up the resources allocated during session establishment. The tool supports both UE-originated (PDU SESSION RELEASE REQUEST) and network-initiated (`Nsmf_PDUSession_ReleaseSMContext`) variants because operators often face abnormal disconnects in the field.

Normal Course of events

PDU Session Release unfolds in six broad phases (see Fig. 3.3 for the detailed sequence diagram):

1. The UE—or the network—sends a PDU Session Release Request. The SMF then initiates SM Policy Association termination and invokes the message of `Nsmf_PDUSession_ReleaseSM` carrying the "release trigger" in subsequent `UpdateSMContext` exchanges with the AMF.
2. The SMF issues an N4 Session Release Request to each UPF; UPFs reply with N4 Session Release Response.
3. After the SMF confirms release status to the AMF, the AMF delivers a NAS PDU Session Release Command to the UE via NGAP.
4. The AMF requests RAN to free AN resources (`N2 Resource Release Request`); the RAN performs any radio-specific cleanup (including forwarding the NAS release), then acknowledges (`N2 Resource Release Ack`).

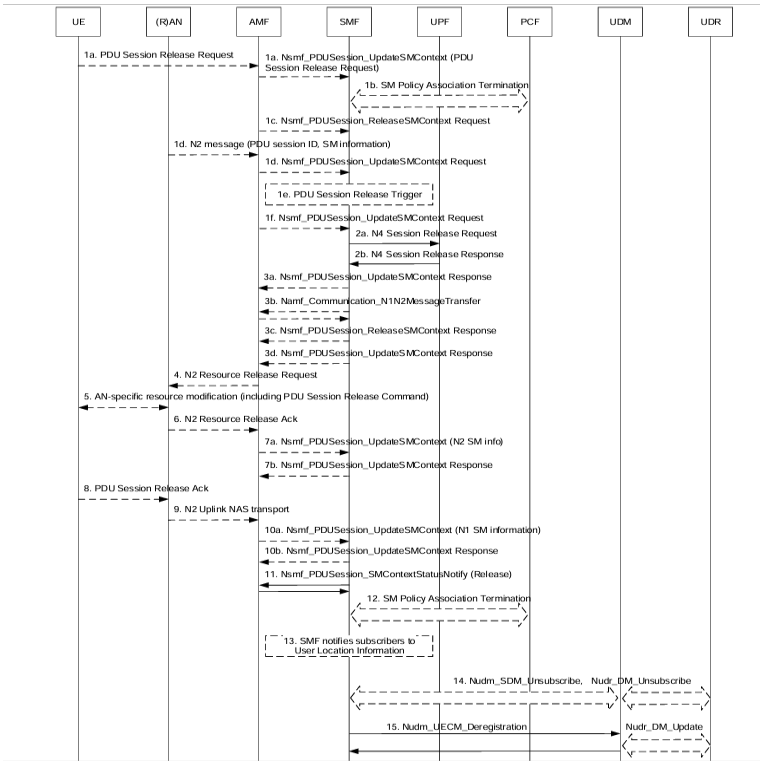

Figure 3.3 PDU Session Release

[6]

5. The UE returns a PDU Session Release ACK. The RAN forwards this to the AMF (`N2 Uplink NAS Transport`), and the AMF and SMF exchange final `UpdateSMContext` messages carrying both N2 and N1 SM information.
6. The SMF sends a `SMContextStatusNotify(Release)` to the AMF, terminates any remaining SM Policy Association with the PCF, and—if configured—publishes UE-location and deregistration notifications to UDM/UDR.

The formation of Client requirements

Functional Requirements:

- The client tool will enable an tester to trigger a network-initiated PDU session release for any active session; it will also detect and display UE-initiated releases.
- Before releasing, it must show the selected session's SMF context (SM Context ID, QoS, tunnel info).

Steps in which the client CLI can participate in the procedure

1. The tester will be able to release a session via command `release-session --id <n>`.
2. The client encapsulates the request in a REST call to the back-end, which maps it to the corresponding UE event.

3. The UE state-event sends a NAS *Release Request*; the core tears down tunnels and policies while the client passively logs progress.

4. After the NAS *Release Complete* is returned, the client confirms successful release and lets the operator create new sessions or proceed with further actions as desired.

3.2.3 Command Context Template

The preceding use-case analysis displays four elementary signaling procedures and, from these, derives the four-tuple command:

```
<node type, node name, command name, flags>

# For each node type, there will have one or many node's names
   . For each node name, there will also contain some kinds of
    command lines

node type >>> node name >>> command --flag
```

Listing 1: Structure of Command Context

At each node type (UE, gNB), there will have a single command `select` to chooose the node name of UE or gNB, for instance, UE *imsi-208930000000001* or gNB *GNB-001*. But, AMF and SMF are called node name there is only one NF specifically

The CLI will follow a hierarchical command structure where each context supports specific commands with configurable flags. The general command syntax follows the pattern:

```
command-name [-flag-name value] [-boolean-flag]
```

Three types of flags are supported throughout the system:

- **String flags**: Accept textual values with optional defaults. Example: `-slice "default"`
- **Integer flags**: Accept numeric values within specified ranges. Example: `-type 1`
- **Boolean flags** : Toggle states without requiring explicit values. Example: `-emergency`

The flag parsing engine supports both GNU-style long flags (`-flag-name`) and positional arguments. When flags are omitted, the system applies predefined default values automatically. For instance, the `create-session` command defaults to slice

"default", data network "internet", and session type 0 if no explicit flags are provided.

Commands operate within a strict node-type hierarchy where each context prefix (ue-XX >>, gnb-YY >>), or even NF context prefix (amf >>, smf >>)identifies both the target component and the specific instance. The client maintains this context through a stack-based navigation model, where the (back, exit) command pops the current level and returns to the parent context. This design enforces type safety by preventing commands intended for UE contexts from being executed in gNB contexts and NF context.

Root context of Core and MSSim

Commands available at the top-level prompt when deploying the ETRIB5G Core and MSSim Client CLI:

- connect <http(s)://host:port>: Establish an HTTP/JSON session to the specified emulator or NF instance. Each context (UE, gNB, AMF, SMF) is reached via its own host and port.
- exit: Terminate the CLI.
- help, clear: Show help or clear the screen.

UE context

Commands for the four signalling procedures are shown in some bullet points:

- register [-emergency]: *Procedure 1 – UE Registration*. The register command initiates the 5G NAS registration procedure. The -emergency flag is a boolean modifier that activates emergency services registration mode. When present, it instructs the UE to request access to emergency services through specialized PLMN selection and registration procedures. This flag corresponds directly to the emergency registration type in the NAS Registration Request message, enabling the UE to establish connectivity even when normal registration is restricted or unavailable.
- deregister [-type <0-5>]: *Procedure 2 – Deregistration*. The deregister command implements UE-initiated deregistration with configurable cause indication. The -type flag accepts integer values from 0 to 3, each corresponding to specific deregistration scenarios mandated by the 3GPP specification. Type 0 represents normal deregistration with the UE remaining reachable for paging; type 1 indicates switch-off deregistration where the UE becomes unreachable; type 2 triggers re-registration required scenarios; and type 3 initiates disable 5GS ser-

vices deregistration. The default value is 0, ensuring graceful disconnection while maintaining network reachability for subsequent procedures.

- `create-session [-slice <sst/sd>] [-dnn <name>] [-type <0|1|2|3>]` *Procedure 3 – PDU Session Establishment*. The `create-session` command establishes PDU sessions with comprehensive parameter control through three distinct flags. The `-slice` flag specifies network slice selection assistance information (NSSAI) in the format `sst[/sd]`, where SST represents the slice/service type and SD denotes the optional slice differentiator. The default value `"default"` instructs the system to use the first available slice from the UE's configuration or AMF-provided allowed NSSAI. The `-dn` flag defines the Data Network Name (DNN) for PDU session context, defaulting to `"internet"` for general-purpose IP connectivity. Alternative values such as `"ims"` or operator-specific DNNs enable access to specialized services. The `-type` flag controls PDU session type selection with values 0 through 3 corresponding to IPv4, IPv6, IPv4v6, and unstructured session types respectively, with 0 (IPv4) as the standard default.
- `back`: Return to server context.

gNB context

RAN-side commands will be displayed with some bullet points below here:

- `release-ue <ue>`: The `release-ue` command requires a mandatory `<ue-id>` argument specifying the target UE identifier. This operation triggers a *UE Context Release Request* to the AMF, initiating the standardised cleanup procedure. The command releases all RAN-side resources including radio bearers, security contexts, and any active PDU sessions associated with the specified UE.
- `release-session <ue-id> <session-id>`: The `release-session` command accepts a mandatory `<ue-id>` parameter and an optional `-id` flag with default value 1. The session identifier must be within the valid range [1,15] as per 3GPP specifications. The `-id` flag allows operators to target specific sessions in multi-session scenarios, while omitting the flag defaults to releasing session ID 1.
- `back`: Return to emulator context.

AMF context

It will automatically enter when connecting to port 9001. The AMF command lines can be listed as some key features below :

- `list-ue`: Display all current UE contexts in the AMF database. Returns JSON-

formatted information including SUPI, SUCI, PEI, TMSI, and MM state for each registered UE. No flags required.

- `view-ue -supi <supi_string>`: View detailed UE context information for a specific UE.
  - `-supi`: *(Required)* SUPI string identifier of the target UE
- `deregister-ue -supi <supi_string> [-cause <0|1|2>]`: Trigger deregistration procedure for a specific UE from the network.
  - `-supi`: *(Required)* SUPI string identifier of the target UE
  - `-cause`: *(Optional)* Integer cause code (0, 1, or 2). Default: 0
- `select-ue -tmsi <tmsi_value>`: Switch to a specific UE context instance for detailed management.
  - `-tmsi`: *(Required)* TMSI value of the target UE context

SMF context

It will automatically enter when connecting to port 9001, same as AMF, but different host. The SMF command lines can be listed as some key features below:

- `list`: Display all PDU sessions in the SMF database. Returns information including SUPI, Session ID, and DNN for each active session. No flags required.
- `release-session -supi <supi_string> -sessionId <session_id>`: Trigger release procedure for a specific PDU session.
  - `-supi`: *(Required)* SUPI string identifier of the UE
  - `-sessionId`: *(Required)* PDU Session ID to be released
- `select-session -supi <supi_string> -sessionId <session_id>`: Switch to a specific PDU session context for detailed management.
  - `-supi`: *(Required)* SUPI string identifier of the UE
  - `-sessionId`: *(Required)* PDU Session ID to select

Table 3.3 summarises the mapping between high-level testing procedures and the concrete commands defined above.

3.3 Component Description

3.3.1 Interactive Tool in 5G Core

In the context of 5G Core, Operations, Administration, and Maintenance (OAM) services are critical for managing and monitoring the network functions. The interactive

Table 3.3 Mapping of procedures to CLI commands

Procedure	Primary Command(s)
Registration	`register` (UE context)
Deregistration	`deregister` (UE context)
PDU Session Establishment	`create-session` (UE context)
PDU Session Release	`release-session` (UE or gNB context)

tool, through its Client CLI, needs to interact with these OAM services to trigger signaling procedures, retrieve network status, and observe real-time events. This interaction is primarily facilitated through well-defined Application Programming Interfaces (APIs), often exposed as HTTP/2-based RESTful services, aligning with the Service-Based Architecture (SBA) of the 5G Core .

Figure 3.4 summarizes the logical relationship between four building blocks: a terminal-based client, an in-process OAM HTTP service, a light plug-in handler, and the network-function state store. The following text reframes their cooperation as an application-layer protocol definition, abstracting away implementation details.

The responsibility of four components in 5G Core can be listed in some bullet points below:

- **Client CLI:** an interactive REST consumer that runs in a terminal.
- **Remote Server:** an HTTP endpoint set embedded in every network function.
- **NF OAM Handler:** translates generic requests into NF-specific operations.
- **State Store:** authoritative data for UEs, sessions and other run-time objects.

With the Transportation from CLI to OAM Service, all traffic flows over HTTP 1.1 with JSON bodies and UTF-8 encoding. Either plain HTTP or HTTPS may be selected according to the deployment environment (bare metal, Docker, Kubernetes, ...). Payloads are intentionally compact so that administrators can reproduce requests with `curl` or similar tools and pipe them through common network tunnels.

API Endpoint

The service exposes exactly three stable resources, each identified by a fixed path rooted under `/oam`:

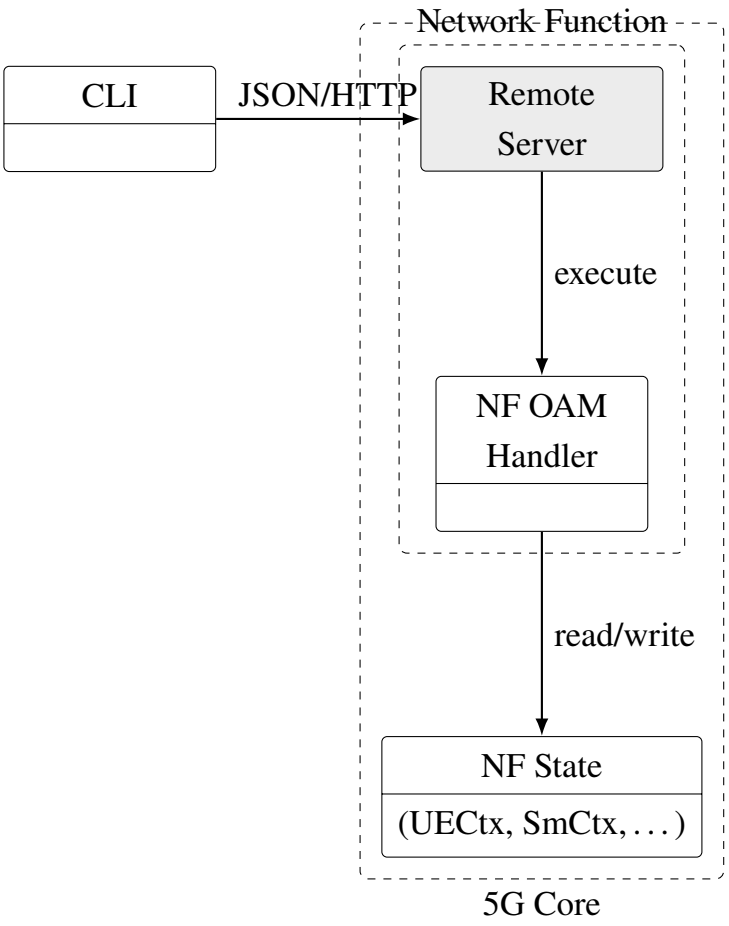

Figure 3.4 The component Interactive Tool in 5G Core

Path	Method	Purpose
`/oam/status`	GET	Liveness and version probe
`/oam/connect`	POST	Capability handshake and context bootstrap
`/oam/cmd`	POST	Stateless execution of follow-up commands

Capability Handshake (`/oam/connect`). The client begins every session by sending the NF identifier, an optional authentication token, and its preferred language/encoding. The service replies with a human-readable banner (prompt string, greeting), a hierarchical catalogue of verbs, sub-contexts, and flag descriptors, and a short-lived context ID that will tag subsequent requests. Because this metadata is generated on demand, the CLI can rebuild its menus dynamically without recompilation whenever the NF's capabilities evolve.

Command Execution (`/oam/cmd`). A request contains `{`

`"ctx": "<id>",`

`"verb": "<operation>",`

`"args": { flag → value }}` The context ID instructs the server which subtree (e.g. specific UE, PDU session, ...) the verb should operate on. The response delivers `code` – transport-level success or failure, `msg` – human-friendly description, and `data` – any structured result produced by the operation. The interface is deliberately stateless apart from the opaque *ctx* token, allowing easy load-balancing or replication of the service component.

Health Probe (`/oam/status`).

A lightweight GET returns an HTTP 200 plus a minimal body containing software version, uptime, and the current timestamp. Platform orchestrators (systemd, Docker, Kubernetes) use this endpoint for readiness and liveness checks.

The error can occur and contain some kind of messages below there:

- HTTP status codes convey transport errors (400 series for client problems, 500 series for server faults).
- An application-level `code` field refines the diagnosis (e.g. "unknown-verb", "permission-denied", "conflict").
- Every error includes a human-readable `msg` and, when appropriate, a machine-parsable `details` object to enable CLI auto-correction or retry.

By limiting the public surface to three fixed endpoints, returning self-describing metadata at connection time, and tagging every command with an opaque context iden-

tifier, the API lets administrators traverse and manipulate the full object space of a 5G core while keeping the protocol lightweight, script-friendly and forward-compatible.

3.3.2 Interactive Tool in Mobile State Simulator

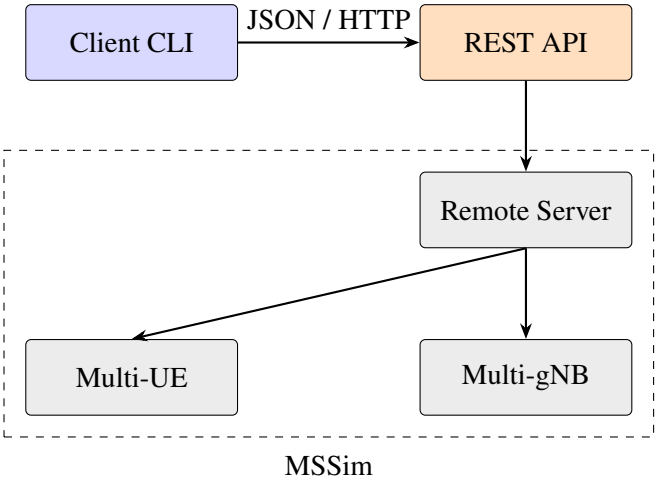

Figure 3.5 Component view of the interactive testing tool.

Figure 3.5 presents the high-level component view of our interactive testing tool. At the top level, users interact via an *Client CLI* which issues JSON/HTTP requests through a lightweight *REST API*. Behind the API gateway lies the *MSSim*, itself composed of a *Remote Server* that exposes the API of UE, gNB and emulator, also maps user commands into protocol events, virtual *Multi-UE* and *Multi-gNB* modules. This division allows each layer to evolve independently—new scenarios, protocols or front-end clients can be added without cross-cutting changes. Below this paragraph is the detailed analysis for the Client-Remote Server:

Client CLI: For the block Client CLI, users type commands in an *ishell*-based session whose prompt reflects the current context. A lightweight context stack help texts and prompt prefixes. Each user action is translated into:

- `GET /api/context` for context discovery,
- `POST /api/context/navigate` for changing context,
- `POST /api/exec` for executing commands.

REST API

The gateway exposes just three endpoints, simplifying versioning and access control. It normalizes JSON payloads and routes them to internal channels; all business logic resides in the MSSim Server.

All traffic between the CLI and the server flows over plain HTTP 1.1 on a sin-

gle TCP port. Requests are sent in JSON format, UTF–8 encoded; responses carry `Content-Type: application/json` and the usual HTTP status codes (`200` success, `4xx` client error, `5xx` server error).

Endpoint Analysis. Three main endpoints are marked as in each paragraph below with their goal of API design in Mobile State Simulator:

`GET /api/context` (connection to Root context of MSSim Server)

The Client CLI returns the list of *top-level* contexts available on the MSSim server. Each entry contains: context type, human-readable name, and a boolean flag telling whether it can be drilled down. The CLI calls this endpoint only once—right after a successful `connect` command—to build the first navigation menu.

`POST /api/context/navigate` (navigation amongs node types)

The Client CLI accepts a JSON body carrying the current context and the navigation verb (*back*, *use*, *select*, *exit*). The server replies with:

- **newContext**: the updated context descriptor for each node type and node name;
- **commands**: all commands valid in that context;
- **prompt**: a short string the CLI can print as shell prompt.

Any navigation error—unknown node, wrong verb, permission denied—is returned as `HTTP 400` plus an `error` field in the JSON body.

`POST /api/exec` (command execution)

The body is a request of command, for instance:

```
{
  "nodeType": "ue",
  "nodeName": "imsi-208930000000001",
  "commandPath": "release-session",
  "args": ["--supi", "010203", "--dnn", "internet"]
}
{
  "nodeType": "gnb",
  "nodeName": "gnb-01-1",
  "commandPath": "release-session",
  "args": ["imsi-208930000000001", "1"]
}
```

The server maps the request to an internal event, executes the corresponding 5G procedure and replies with a response, for example:

```
{
  "response": "PDU session 1 established (10.0.0.8)",
  "error":    ""
}
{
  "response": "PDU session 1 released (10.0.0.8)",
  "error":    ""
}
```

For the error handling, the API endpoint will notify in the logs of the backend Remote Service. These errors below can be listed:

- Network failures are detected via HTTP status and surfaced to the user without stack traces (*"connection refused"*, *"timeout"*).
- Semantic errors inside a command are reported in the JSON `error` field
- The transport is stateless; if the CLI crashes, reconnecting restores the previous working state.

By encapsulating every interaction in the three endpoints above, the interactive tool decouples the user interface from the core simulator and creates a clean seam for additional Client CLI without disturbing the underlying 5G signalling logic.

3.4 Integration Strategy

The interactive tool integrates with the ETRIB5G Core system through a multi-layered approach that preserves the separation between signaling control and data plane operations while providing comprehensive testing capabilities. The integration strategy follows a systematic workflow designed to ensure reproducible testing environments and reliable diagnostic access to network function internals.

Architectural Integration Approach The Client CLI - Remote Server integration operates on three fundamental principles: embedded instrumentation, service mesh compatibility, and protocol transparency. Each network function incorporates OAM capabilities through standardized Go interfaces, enabling runtime introspection without modifying core 5G protocols. The HTTP-based OAM endpoints coexist with existing SBI interfaces, allowing simultaneous production traffic and diagnostic operations. This approach ensures that testing activities do not interfere with normal network function behavior while providing deep visibility into signaling procedures.

MSSim Emulator Integration The system integrates with UE and gNB simulators through shared memory contexts and event-driven state machines. When the CLI triggers operations such as UE deregistration or session release, these commands directly manipulate the same data structures used by the live simulation, ensuring immediate propagation of test actions. The simulator maintains separate control channels for test orchestration while preserving the timing-critical aspects of 5G procedures, enabling realistic testing scenarios that accurately reflect air interface behavior.

The integration workflow in Fig. 3.6 follows a linear progression where each step establishes the foundation for subsequent operations, ensuring systematic problem detection and isolation.

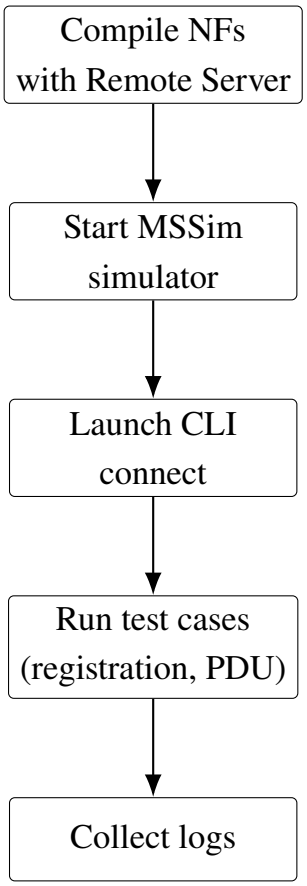

Figure 3.6 Flow diagram of the integration strategy.

The workflow begins with compilation of network functions that include embedded OAM Handlers, ensuring that diagnostic capabilities are built into the system from the ground up. During deployment, each network function automatically exposes both its standard 5G interfaces and additional OAM endpoints, typically under the ‘/oam/*‘ path structure.

The MSSim simulator initialization establishes the test environment with configurable scenarios including registration procedures and PDU session management The

simulator operates with precise timing control and can generate realistic network conditions, providing a controlled environment for signaling procedure validation.

CLI connection establishment creates the interactive testing session, where testers can navigate through network function contexts and execute real-time commands. Finally, in the client, testers will be announced with the message from the client, and also in the backend server, the logs will be displayed in full screen.

3.5 Implementation

3.5.1 Development Environment

The Interactive Tool (client) comprises a client-server architecture implemented entirely in Go, leveraging Golang's built-in concurrency primitives and cross-platform compatibility for seamless operation across Windows and Linux environments.

Client-Side Implementation. The CLI client utilizes the `abiosoft/ishell v2.0.0` library to provide a full-featured interactive terminal interface with command history, context-aware prompting, and dynamic menu generation. The client communicates with network functions through standard HTTP/JSON APIs, enabling both manual interaction and programmatic automation via tools like `curl`.

Server-Side Backend. Each network function embeds an HTTP service built on the Gin web framework (`gin-gonic/gin v1.9.0`). Command definitions and flag parsing leverage `urfave/cli/v3 v3.3.3`, which automatically generates help text and validates arguments while maintaining type safety across the API boundary.

Build System and Deployment. The build process is orchestrated through a single Makefile target that compiles the standalone CLI client binary. The same Go source tree produces native executables for both development workstations and container environments without platform-specific modifications, ensuring consistent behavior across deployment scenarios.

3.5.2 Implementation Details-Interactive CLI in 5G Core

Figure 3.7 illustrates the comprehensive package-layer architecture spanning from the client-side interactive shell to the deep state manipulation within individual network functions. The design employs strict interface separation and dependency injection patterns to achieve modularity and extensibility across the entire 5G core system.

Client-Side Package Structure. The CLI client comprises three core modules: `Client Core` implements the main Client struct with HTTP communication methods, `Context Manager` manages hierarchical command contexts with parent-child relationships, and

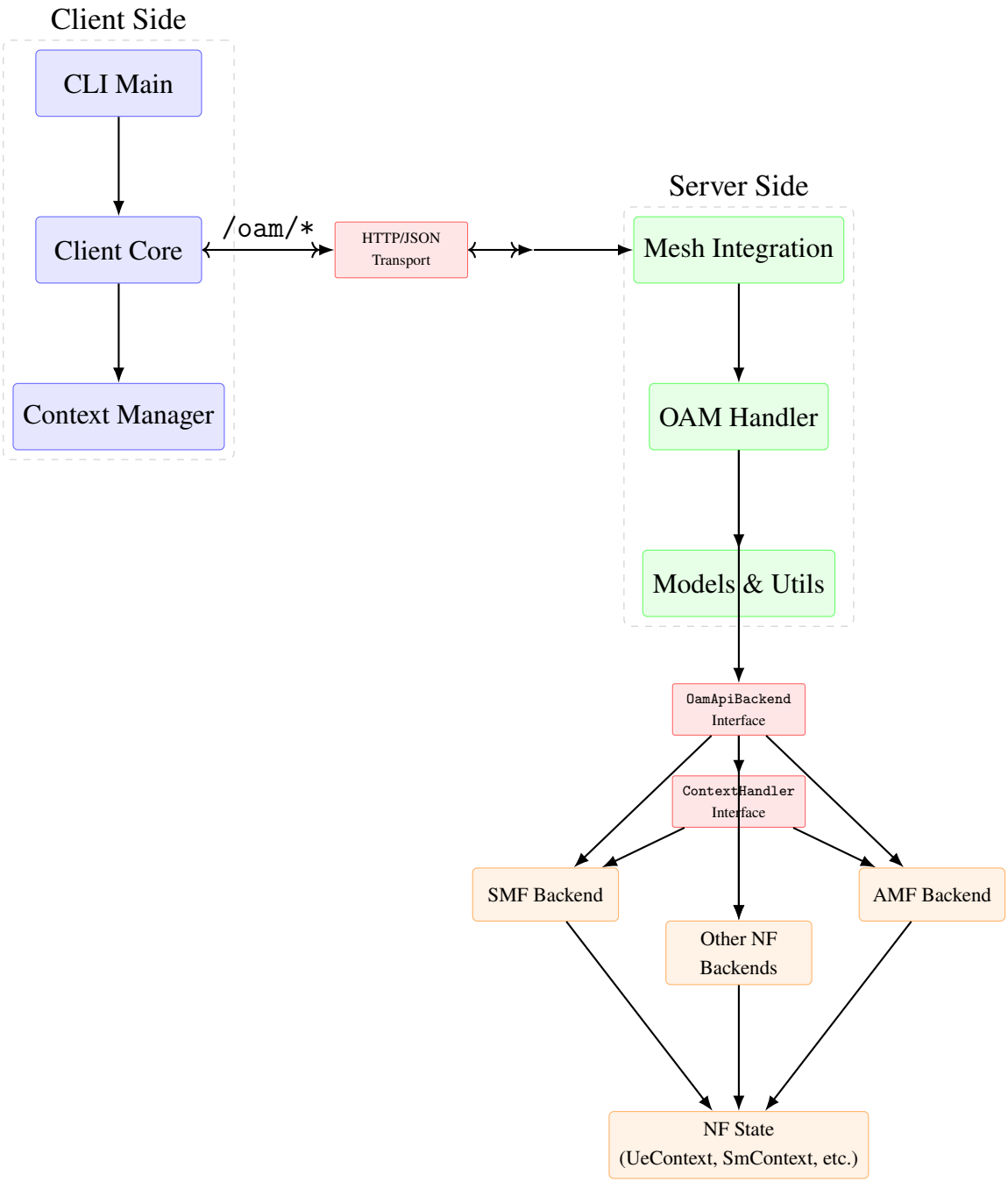

Figure 3.7 Package Layer Architecture of Client/OAM Server in ETRIB5GC

`CLI Main` provides the application entry point. The Client maintains an internal Context stack that dynamically rebuilds command menus based on server responses, enabling seamless navigation between network function scopes (e.g., AMF $\rightarrow$ UE $\rightarrow$ Session) without client recompilation.

Server-Side OAM Framework. The server framework defines two fundamental interfaces: `OamApiBackend` for network function integration and `ContextHandler` for command execution. The `OamHandler` struct implements HTTP route management by exposing exactly three REST endpoints (`/oam/status`, `/oam/connect`, `/oam/cmd`) through the Gin web framework. Each network function embeds an OAM backend by implementing the `HasOamBackend` interface, which the mesh framework automatically detects during service initialization and registers the appropriate HTTP routes.

Network Function Integration Pattern. Individual network functions implement OAM support through dedicated `oam-backend` packages. Each backend defines command maps using `urfave/cli/v3` Command structs, which is automatically converted into JSON-serializable command's information structures for client consumption. Context handlers leverage Go's context-value mechanism to inject handler instances into CLI command actions, enabling type-safe access to network function state during command execution.

Dynamic Context Management. The system implements hierarchical context switching through opaque context identifiers (e.g., "amf", "ue:imsi-2089300000001", "session:001:1"). The server maintains context-specific command handlers that are instantiated on-demand through the `GetHandler(contextId)` method. When commands trigger context switches (e.g., selecting a specific UE), the server returns a new `ServerContext` containing updated command metadata, prompting the client to rebuild its interactive menu structure automatically.

Command Processing Pipeline. Command execution follows a standardized pipeline: (1) Client serializes command name, context ID, and arguments into JSON command-request, (2) Server routes the request to the appropriate context handler based on context ID parsing, (3) Handler invokes the corresponding `cli.Command` with a captured output buffer, (4) Response contains either result text or a new context descriptor for navigation. This design ensures that all command logic remains within network function modules while providing consistent HTTP transport semantics.

Error Handling and Type Safety. The implementation employs multiple error han-

dling layers: HTTP status codes for transport issues, application-level error fields in JSON responses for command failures, and Go's built-in error propagation for internal processing. Type safety is maintained through interface contracts and compile-time validation of command structures, while runtime command discovery prevents client-server version mismatches by dynamically adapting to available functionality.

3.5.3 Implementation Details-Interactive CLI in Mobile State Simulator

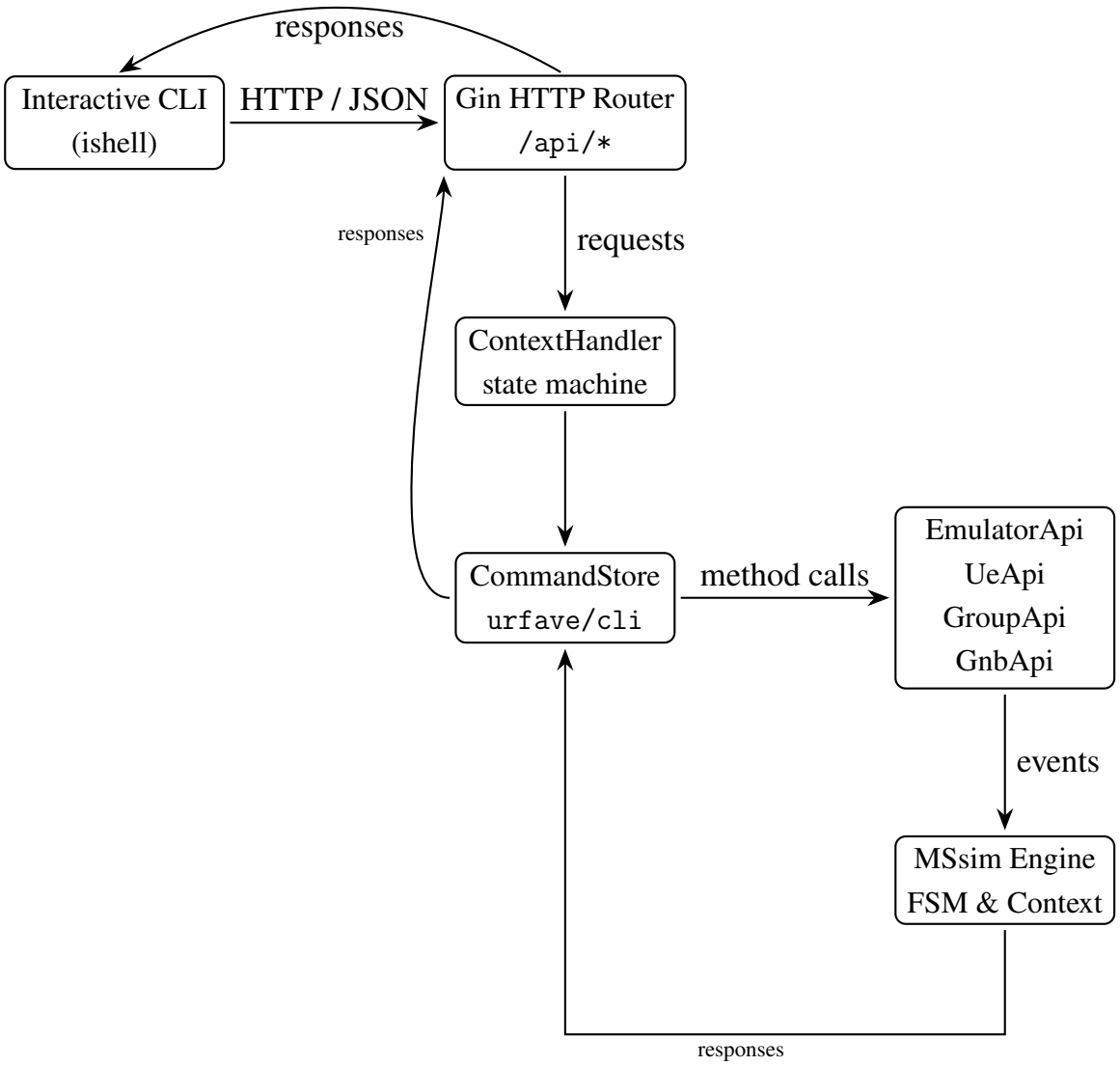

Figure 3.8 Block diagram of the client–server signalling path within MSSim.

This section provides an in-depth discussion of the design and operation of the interactive client–server tool that integrated with MSSim. The architectural overview shown in Figure 3.8 serves as a visual anchor: each paragraph below follows the direction of the arrows in the diagram, beginning at the user's keyboard and ending inside the finite-state machines that drive the core of the simulator.

Client-Side Implementation:

The client leverages the `abiosoft/ishell` library to provide a command-line experience. The core `Client` struct maintains an `Shell` instance alongside a context stack implemented as a slice of `ClientContext` objects. Each context encapsulates its type (root, server, emulator, node), available commands, and hierarchical relationships, allowing the shell to rebuild its command map dynamically as users navigate between different operational scopes.

Context transitions are managed through a finite state model where the client sends `NavigationRequest` payloads to the server's `/api/context/navigate` endpoint. Upon receiving a `NavigationResponse`, the client updates its local context stack, refreshes the command completion database, and adjusts the shell prompt to reflect the new operational mode.

Server-side Implementation

REST API The server exposes its functionality through a minimal HTTP interface built with the `gin-gonic/gin` framework. The gateway layer is deliberately stateless—it performs only JSON unmarshalling, request routing, and response formatting. Session state is maintained in the client's context stack and in the server's finite-state machine instances, ensuring that the gateway can be horizontally scaled or replaced without affecting ongoing test sessions.

Three endpoints handle all interactions: `GET /api/context` returns the top-level context catalogue, `POST /api/context/navigate` processes hierarchical navigation, and `POST /api/exec` dispatches commands to the appropriate backend modules. Each endpoint validates its input schema and returns structured error responses, making the API suitable for both interactive use and programmatic access from CI/CD pipelines.

Context Handler The `ContextHandler` maintains a hierarchical map of available contexts and manages navigation between them. At startup, it constructs a tree structure representing the complete operational scope: root context, server context, context sets (emulator, UE, gNB), and individual node contexts.

Navigation logic implements a stack-based model mirroring the client side. When processing `use`, `select`, `back`, or `disconnect` commands, the handler validates transitions, updates the context hierarchy, and returns appropriate command lists for the target context. This centralized navigation logic ensures consistent behaviour regardless of whether users access the system through the CLI, web interfaces, or API calls.

Command Store The `CommandStore` acts as a command registry and execution engine. At initialization, it constructs `cli.Command` trees for each node type (emulator,

UE, gNB, group) using the `urfave/cli/v3` framework. These command definitions include flag specifications, argument validation, and help text generation, providing a uniform interface regardless of the underlying implementation.

Command execution follows a channel-based pattern where each command receives a response channel through its execution context. The command implementation writes its output to this channel, decoupling command logic from HTTP response handling. This design allows commands to stream long-running outputs (such as registration progress for large UE populations) without blocking the HTTP gateway.

Remote API Layer

Protocol Translation The remote layer serves as a translation boundary between the HTTP-based external interface and the internal finite-state machine architecture. Four API interfaces (`EmulatorApi`, `UeApi`, `GnbApi`, `GroupApi`) define the operations available for each node type, while their implementations handle the conversion from REST semantics to FSM events.

Object Lifecycle Management Each API implementation maintains runtime inventories of active objects (UEs, gNBs, groups) and provides discovery methods (`ListUes()`, `ListGnbs()`) that the context handler uses to validate `select` commands. Object creation and destruction are handled through factory methods that coordinate with the core FSM engine to ensure consistent state management.

The remote layer also implements an event forwarding mechanism that allows external systems to inject events into running simulations. This capability enables advanced testing scenarios where external scripts can trigger handovers, inject faults, or simulate load patterns that complement the interactive command interface.

This layered architecture ensures that changes to the user interface, transport protocol, or core simulation engine can be made independently, while the well-defined package boundaries provide clear testing and integration points for each component.

CHAPTER 4. TESTING AND EVALUATION

4.1 Setup the environment

This section describes the comprehensive setup and execution procedures for deploying the ETRIB5GC network and Mobile State Simulator with the integrated interactive testing tool by the order of number, as shown in figure 4.1. The setup encompasses the complete 5G core network deployment, MSSim emulator configuration, and OAM client initialization in a local development environment.

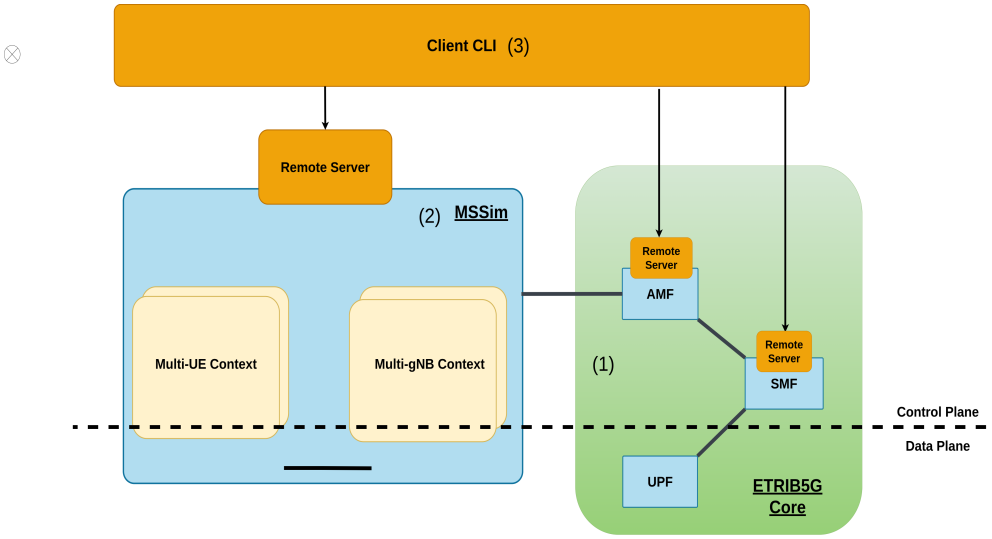

Figure 4.1 Order of System to be setup

System Prerequisites and Environment

The testing environment requires a Linux-based system - Ubuntu 20.04 LTS with Linux kernel version 5.4.x to ensure compatibility with the gtp5g kernel [7] module dependency , MongoDB database server, and sufficient network interface configuration for multi-instance deployment. The system utilizes localhost loopback interfaces with distinct IP addresses for each network function to simulate distributed deployment while maintaining local accessibility. Docker and containerization tools are optional but recommended for isolated testing environments.

The development environment is configured with distinct host and port assignments for each network function to ensure isolation and facilitate testing. The controller service runs on a dedicated host at port 8888. The AMF is configured to bind to port 9001 for both its SBI and OAM interfaces. Similarly, the SMF also utilizes port 9001 on its respective host, while the UPF is deployed on a separate host to ensure data plane isolation. This structured addressing and port allocation scheme enables clear separation

between network function instances and supports comprehensive inter-NF communication testing.

4.1.0.1 Setup ETRIB5G Core (1)

Network Function Compilation and Build Process

The ETRIB5GC compilation process utilizes a centralized Makefile that orchestrates the building of all network functions with embedded OAM capabilities, as conducted in Listing 1 below. The build system ensures that each network function includes the necessary OAM plug-ins and HTTP service endpoints required for interactive testing support.

```
# Build all network functions with OAM support
make all

# Individual network function compilation
make controller  # Service mesh controller
make amf         # Access and Mobility Management Function
make smf         # Session Management Function
make upf         # User Plane Function
make nsm         # Network Slice Management
make udm         # Unified Data Management
make ausf        # Authentication Server Function
make pcf         # Policy Control Function

# Build interactive CLI client
make oam
```

Listing 2: Network Function Build Process

The compilation process produces statically-linked binaries, each incorporating the mesh service framework and OAM interface capabilities. The build system automatically includes all necessary dependencies and ensures cross-platform compatibility for both development and deployment environments.

Configuration Management

Network function configuration utilizes JSON-based configuration files, providing centralized management of PLMN identifiers, network slice configurations, and service mesh parameters. Each network function maintains its own configuration file with specific parameters for mesh integration, OAM interface binding, and functional behavior.

The AMF configuration specifies PLMN ID (MCC: 208, MNC: 93), AMF set

identifier (10-100), service mesh registration parameters, and OAM interface binding (port 9001). The SMF configuration includes network slice specifications (SST: 1, SD: 010203) and corresponding mesh service labels for proper service discovery and routing.

```
{
  "plmnId": {
    "mcc": "208",
    "mnc": "93"
  },
  "amfSet": "10-100",
  "mesh": {
    "registrarIp": "127.0.0.2",
    "bind": "127.0.0.9"
  },
  "oam": {
    "bind": "127.0.0.9",
    "port": 9001
  }
}
```

Listing 3: AMF Configuration Example

Service Mess Deployment

The deployment process begins with the service mesh controller initialization, which provides service discovery, load balancing, and inter-NF communication management. The controller configuration (`config/controller.json`) defines service selectors, routing policies, and stateful service management for all network functions.

```
# Start service mesh controller
./bin/controller -c config/controller.json

# Deploy core network functions
./bin/nsm -c config/nsm.json &
./bin/udm -c config/udm.json &
./bin/ausf -c config/ausf.json &
./bin/pcf -c config/pcf.json &
./bin/amf -c config/amf.json &
./bin/smf -c config/smf.json &
./bin/upf -c config/upf.json &
```

Listing 4: Service Mesh Deployment Sequence

Each network function automatically registers with the service mesh controller

upon startup, establishing service discovery relationships and enabling dynamic routing configuration. The mesh framework provides automatic health checking, service registration, and inter-NF communication management without requiring manual configuration.

4.1.0.2 Setup MSSim (2)

MSSim emulator configuration

```
gnodeb:
  controlif:
    ip: "127.0.0.x"
    port: 9487
  dataif:
    ip: "127.0.0.x"
    port: 2152
  listGnbs:
    - plmnlist: # gnb 1
        mcc: "208"
        mnc: "93"
        tac: "000001"
        gnbid: "000008"
      slicesupportlist:
        sst: "01"
        sd: "010203" # optional, can be removed if not used
defaultUe:
  msin: "9985507341"
  key: "efa44b14761d44b4b70adb371643ecfe"
  op: "fb6d48c2591e3c31a6db7a0ae4efde91"
  amf: "8000"
  sqn: "00000000"
  dnn: "internet"
  routingindicator: "0000"
  hplmn:
    mcc: "208"
    mnc: "93"
  snssai:
    sst: 01
    sd: "010203" # optional, can be removed if not used
  integrity:
    nia0: false
    nia1: false
    nia2: true
```

```
    nia3: false
  ciphering:
    nea0: true
    nea1: false
    nea2: true
    nea3: false
amfif:
  - ip: "127.0.0.x"
    port: 38412
scenarios:
  - nUEs: 1 # group UEs 1
    gnbs:
      - "000008"
    ueEvents:
      - event: register
        delay: 2
        register_type: 0
      - event: sessioncreate
        delay: 3
        pdu_session_type: 0 # 0: Initial; 1: Emergency
remote:
  enable: true
  ip: "0.0.0.0"
  port: 4000
```

Listing 5: MSSim emulator configuration

The MSsim framework is configured using YAML files, where the primary configuration file specifies essential network parameters. These include the addressing for the gNodeB's control interface (N2) and data interface (N3), along with the endpoint details for connecting to the AMF. The framework also defines UE authentication parameters, including IMSI components (MCC, MNC, MSIN), cryptographic credentials (K, OPc, AMF, SQN), and slice selection identifiers (SST, SD). This structured configuration enables accurate emulation of signaling and data paths within the simulated 5G environment.

Network interface and addressing setup

The test environment utilises a bridge network configuration where both MSsim and ETRIB5GC components communicate over the 127.0.0.x address. This addressing scheme enables direct communication between simulated RAN components and core network functions without requiring complex routing or network address translation.

MSsim compilation and execution

The MSsim framework is compiled using the provided Makefile with the `make` command, which invokes the Go build system to produce a single statically-linked binary executable.

The MSsim emulator is subsequently launched with specific test scenario parameters, supporting single UE testing custom scenario execution (`sudo ./mssim scenarios -custom ./config/custom.yml`).

4.1.0.3 Setup Client (3)

Client compilation with MSSim

The interactive CLI client is integrated within the MSsim and activated through command-line flags or configuration file settings. The remote server component exposes RESTful APIs on configurable ports (default 4000 for MSsim) to enable external tool integration. The client establishes HTTP connections to these endpoints and utilises JSON-based command serialisation for procedure triggering and status monitoring.

```
# Launch interactive CLI client
./client

# Connect to MSSim Server interface
>>> connect http://localhost:4000

# Interactive command execution
>>> use ue
>>> select imsi-208930000000001
imsi-208930000000001>>> register --emergency
imsi-208930000000001>>> create-session --slice default --dn
   internet --type 1
```

Listing 6: MSSim Client Deployment and Usage

Client compilation with ETRIB5GC

The client CLI deployment provides the interactive testing interface for network function management and signaling procedure validation. The CLI client operates as a standalone application that establishes HTTP connections to network function OAM interfaces, enabling real-time testing and monitoring capabilities.

```
# Launch interactive CLI client
./bin/oam-cli
```

```
# Connect to AMF OAM interface
>>> connect http://127.0.0.9:9001

# Connect to SMF OAM interface
>>> connect http://127.0.0.10:9002

# Interactive command execution
amf> list-ue
amf> select-ue --tmsi 0x12345678
ue:12345678> view
ue:12345678> deregister --cause 1
```

Listing 7: OAM Client Deployment and Usage

4.2 Test Scenarios

4.2.1 Execution Workflow and Testing Procedures

The complete testing workflow follows the 3.4 outlined in Chapter 3, beginning with network function startup verification and progressing through UE registration, session establishment, and signaling procedure testing. Each phase includes validation checkpoints to ensure proper system operation before advancing to the next testing stage.

Test execution procedures leverage the interactive CLI capabilities to trigger specific signaling events, monitor real-time system responses, and collect performance metrics throughout the testing process.

4.2.2 UE Registration

4.2.2.1 Test scenario overview

The registration test simulates a UE transitioning from the *5GMM-NULL* state to *5GMM-REGISTERED* through the standardised 3GPP procedure. The interactive Client CLI - Remote Server tool initiates this process through a simple command like `imsi-208930000000001» register`, which triggers the internal finite-state machine to generate the appropriate NAS messaging sequence, as shown in figure 4.2.

4.2.2.2 UE Registration From MSSim

The operator initiates the registration sequence by connecting to the emulator server (`connect http://localhost:4000`), navigating to a specific UE context (`use ue; select ue-01`), and issuing the registration command `register -emergency`. The client immediately translates this into a command request that is posted to the `/api/exec` endpoint, where the server's command handler maps it to an internal `RegisterInit` event within the UE's state machine.

This aspect of the test is particularly valuable for identifying integration issues between different core network functions. The tool can help tester to verify that UE was completely registered to Core Network or not, which the AMF successfully registers a UE but fails to establish proper policy contexts, which would lead to service degradation in operational networks.

4.2.2.3 Test validation and metrics

The registration test concludes with comprehensive validation of the final network state. The tool verifies that the UE has transitioned to the `5GMM-REGISTERED` state, that appropriate GUTI allocation has occurred, and that the AMF has established complete UE context information. Additionally, the test validates that any requested PDU session contexts have been properly initialised and are ready for subsequent session establishment procedures.

Performance metrics collected during the test include registration latency (time from initial request to final accept message). These metrics provide quantitative assessment of network performance and can be used to identify configuration issues that impact user experience.

The test framework also validates error handling by introducing controlled failures at various points in the registration flow. For example, the tool can simulate registration failures, or policy server unavailability to verify that the UE and AMF respond appropriately according to the specifications.

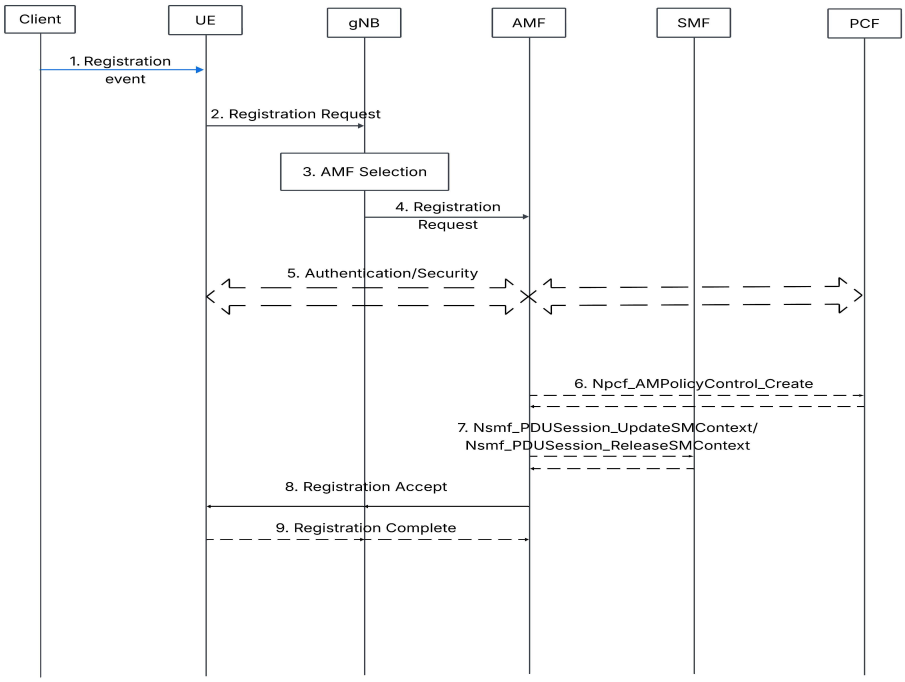

Figure 4.2 UE Registration Test case

4.2.2.4 *Result of Testing*

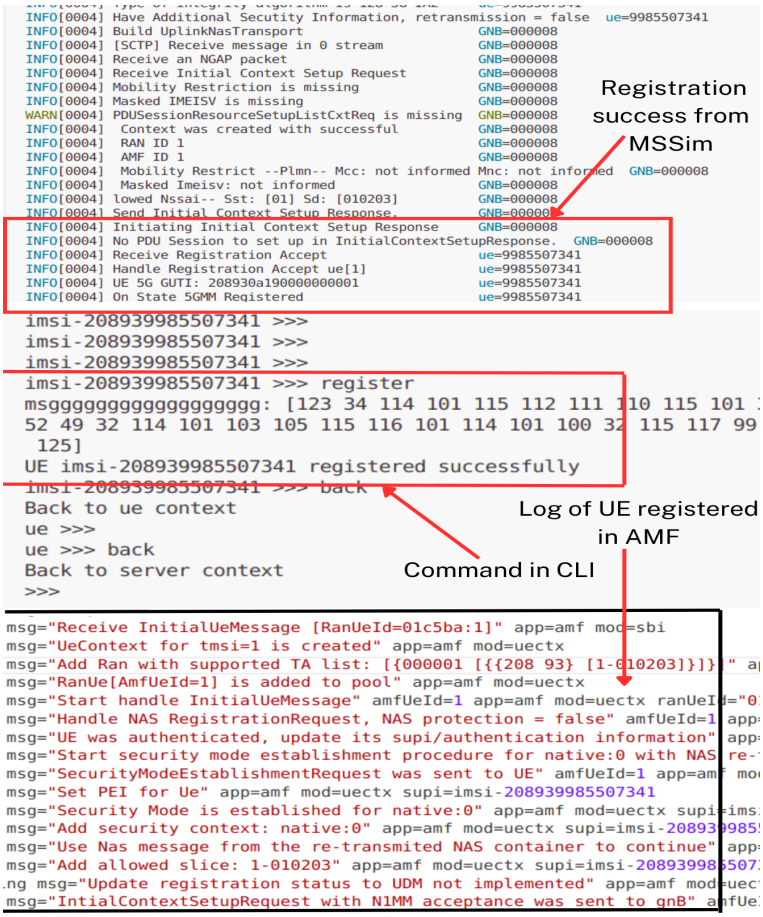

Figure 4.3 Result of UE Registration Success into Core Network

The figure 4.3 illustrates the successful completion of the UE Registration Signaling Procedure using the interactive CLI tool integrated into the Mobile State Simulator (MSSim) and the ETRI B5GC system. From the log output, we observe that the UE with *imsi-208939985507341* initiated a registration request via the command `register` through the CLI.

This request triggered the registration signaling flow between the UE and the 5G core network, as reflected in the lower log section, which shows detailed NAS and NGAP messaging handled by the AMF. Messages from logs of AMF and MSSim such as `Receive InitialUeMessage`, `SecurityModeEstablishmentRequest`, and `Registration Accept` confirm that the UE underwent the standard 5G registration steps. The logs also show the assignment of a `GUITI (208930a19000000001)` and the transition of the UE state to `5GMM Registered`. These outputs validate the correct functioning of the simulated UE registration procedure using the MSSim CLI and confirm successful interaction with the AMF in the ETRI B5GC architecture.

4.2.3 UE Deregistration

4.2.3.1 Test Scenario Overview

UE deregistration testing focuses on the AMF's role as the mobility management anchor point, where the interactive tool can initiate deregistration procedures that cascade through the entire 5G system architecture. The test validates both the immediate signaling response and the subsequent resource cleanup operations that span across AMF, SMF, and UPF network functions.

4.2.3.2 Deregistration From Mobile State Machine

In network-initiated scenarios, the test framework simulates AMF-driven deregistration by leveraging MSsim's internal finite state machine engine to generate appropriate NAS signalling. The AMF context within MSsim maintains active UE registrations and can selectively trigger deregistration procedures based on configurable policies such as idle timeout, resource constraints, or administrative commands. The deregistration test begins with the CLI client establishing a connection to the remote server in MSSim, which provides direct access to the UE context. Through this interface, the test framework can inspect and manipulate UE states, allowing the control over when and how deregistration is initiated.

4.2.3.3 Deregistration From AMF Trigger Mechanism

The deregistration process can also begin when the CLI client sends a command to the AMF context via the OAM interface into 5G Core, specifying the target UE and an optional cause code that determines signaling behavior. This command is translated by the interactive tool into internal AMF state machine events that emulate network-initiated deregistration. The AMF updates the UE's mobility state and triggers cleanup procedures, including releasing PDU sessions in coordination with the SMF. Testers can select UEs using identifiers such as SUPI or TMSI and initiate deregistration with precision, enabling targeted testing within multi-UE scenarios.

4.2.3.4 End-to-End Signaling Validation

The complete deregistration test scenario validates the end-to-end signaling flow from the initial CLI command through the final cleanup confirmation across all involved network functions. The test verifies that the deregistration acceptance message properly reaches the UE simulator. However, as indicated in the sequence diagram figure 4.4, the normal deregistration accept message flow may be interrupted or modified for testing purposes, allowing evaluation of error handling and timeout scenarios.

Test operators can observe the progression of state changes across AMF UE con-

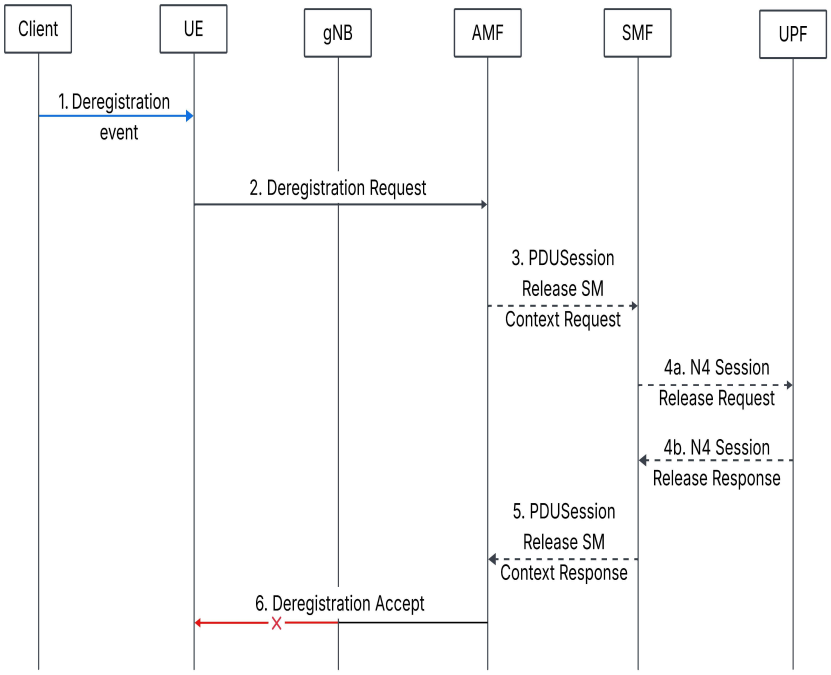

Figure 4.4 UE Deregistration Test case

texts, SMF session contexts, and UPF forwarding rules, ensuring that the deregistration procedure maintains system consistency and proper resource management.

4.2.3.5 Result of Test

The first figure 4.5 demonstrates the successful execution of the UE Deregistration procedure, initiated either directly from the AMF command-line interface (CLI) or through the CLI of the MSSim.

In this scenario, the UE with *imsi-208939985507341* had already completed the registration process and established a PDU session with the 5G core. The deregistration command (`deregister -type 0`) issued from MSSim triggers the UE to send a NAS Deregistration Request to the AMF. The corresponding AMF logs show the handling of this request, including context deletion, session release notification, and the removal of the UE context. If the deregistration is initiated from the AMF CLI using the deregister-ue command, a similar sequence is observed. In both cases, the deregistration is executed successfully, and the UE transitions to an unregistered state while its session-related resources begin to be released.

The second figure 4.6 highlights the internal core network response following the UE deregistration, specifically focusing on session release procedures in the SMF and UPF. After the AMF receives the UE deregistration request, it notifies the SMF, which then initiates the session release by sending a `ReleaseSmContextRequest`. The SMF log confirms the deletion of the PDU session context associated with the UE, including

Figure 4.5 Result of UE Deregistration Success out of Core Network

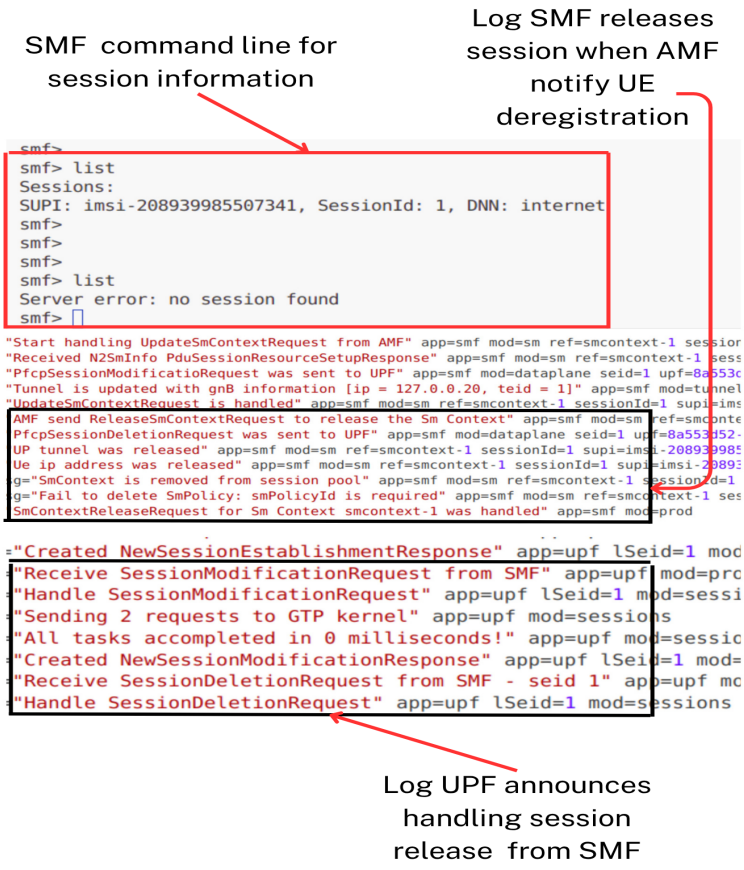

Figure 4.6 Result of UPF and SMF release session with UE Deregistration

PFCP session deletion requests sent to the UPF.

Simultaneously, the UPF log shows that it receives and handles the message of the `SessionDeletionRequest` from the SMF, effectively releasing the data tunnel and session resources. This behavior confirms that the system reacts promptly and correctly to deregistration events—automatically cleaning up the data and control plane sessions, even when a session has already been established prior to deregistration.

4.3 Evaluation Criteria

The evaluation of the interactive tool for testing signaling procedures in 5G Core Platform focuses on four key criteria: responsiveness, flexibility, ease of use, and fault detection capability. These criteria directly address the testing challenges outlined in Subsection 2.4 and validate the achievement of the thesis objectives.

Responsiveness and Performance. The HTTP-based client-server architecture demonstrates sub-millisecond response times for command execution and state queries. The `ishell` framework provides immediate feedback with real-time context switching between network functions. For example, task handling with timeouts ensures long-running operations like UE deregistration do not block the interface. Timer-based procedures maintain millisecond precision, enabling accurate 5G signaling protocol validation across AMF, SMF, and UPF instances.

Flexibility and Extensibility. The modular client-server backend architecture enables seamless integration of new network functions and simulator through the context handler interface pattern. Each NF implements domain-specific commands via the standardized `cli.Command` framework while maintaining consistent user interaction. The hierarchical context system allows navigation from global AMF context to specific UE contexts, with dynamic command sets reflecting current testing scope. JSON-based messaging ensures cross-vendor compatibility and external framework integration.

Ease of Use and User Experience. The context-aware CLI significantly reduces complexity barriers through intuitive navigation and automatic command discovery. Complex scenarios like UE registration execute through simple command sequences. Automated deviation flagging transforms manual correlation into automated validation.

Areas for improvement include automated test result archiving and enhanced scalability monitoring for large-scale UE testing scenarios. However, the tool effectively addresses productivity challenges by enabling testers to focus on scenario design rather than data gathering tasks.

CONCLUSION

General Conclusion

As mobile networks evolve toward beyond 5G and 6G, open-source platforms play a critical role in enabling rapid experimentation and innovation. Their flexibility allows researchers to prototype and validate complex signaling procedures—an essential aspect of maintaining performance and stability in 5G systems, where interactions span both UE and core network functions.

This thesis introduced an interactive client-server tool for testing signaling procedures in open-source 5G environments. The system includes a CLI-based client and a server module integrated with core components and simulators, allowing users to trigger, monitor, and analyze flows such as registration and deregistration.

Evaluation with typical scenarios confirmed the tool's ability to simulate both UE-initiated and network-initiated procedures. Its interactive design supports repeatable and controlled testing, making it suitable for research, debugging, and educational use.

Potential Extensions

While the current tool provides a solid foundation, several extensions could enhance its capabilities. These include broader protocol coverage such as handover, paging, and additional layers like NGAP and PFCP; support for automation through scripting or a GUI front-end to improve usability; integration of real-time metrics and visualization for performance monitoring; and adaptation to emerging 6G technologies such as AI-driven networking and non-terrestrial networks, ensuring continued relevance in future research.

REFERENCES

[1] Q. T. Thai and N. Ko, "Towards realizing a cloud-native b5g mobile core architecture," in *Proceedings of the 2022 13th International Conference on Information and Communication Technology Convergence (ICTC)*. IEEE, 2022, pp. 1018–1023.

[2] "General 5g core architecture," https://www.geeksforgeeks.org/computer-networks/5g-network-architecture/, accessed: June 22, 2025.

[3] S. Rommer, P. Hedman, M. Olsson, L. Frid, S. Sultana, and C. Mulligan, "Network functions and services," in *5G Core Networks: Powering Digitalization*. Academic Press, 2020, ch. 13. [Online]. Available: https://doi.org/10.1016/B978-0-08-103009-7.00013-2

[4] 3rd Generation Partnership Project (3GPP), "5G; 5G system; principles and guidelines for services definition; stage 3," 3GPP, Tech. Rep. TS 29.501, v16.0.0, Nov. 2020, [Online]. Available: https://www.etsi.org/deliver/etsi_ts/129500_129599/129501/16.04.00_60/ts_129501v160400p.pdf.

[5] S. Rommer, P. Hedman, M. Olsson, L. Frid, S. Sultana, and C. Mulligan, "Selected call flows: Registration, deregistration, pdu session establishment/release, inter-ng-ran handover," in *5G Core Networks: Powering Digitalization*. Academic Press, 2020, ch. 15. [Online]. Available: https://doi.org/10.1016/B978-0-08-103009-7.00015-6

[6] "5G System; Procedures for the 5G System; Stage 2," ETSI, Tech. Rep. TS 123 502 V17.7.0, Jan. 2023, section 4.3.4.2: "PDU Session Release". [Online]. Available: https://cdn.standards.iteh.ai/samples/67681/0a0f2faededb4d0ab3fae55ae6ad6a2c/ETSI-TS-123-502-V17-7-0-2023-01-.pdf

[7] free5GC Project, "gtp5g: A linux kernel module for gtp-u in 5g core network," https://github.com/free5gc/gtp5g, 2024, accessed: 2025-06-26.